

Spatial Epidemiology in R: A Hands-On Tutorial

**Ghana R User Community — R-Only Track. Prepared for the Ghana R User
Community & AfreDAC Collaboration**

George K. Agyen - Ghana R User Community

2026-08-05

Table of contents

Welcome	4
What You Will Learn	4
Prerequisites	5
How to Use This Tutorial	5
Key Reference	5
Let's Begin!	5
1 Module 1: R Environment Setup and Spatial Data Import	6
1.1 Install and Load R Packages	6
1.2 Load Administrative Boundary Data	8
1.3 Importing Your Own Spatial Data	9
1.4 Load Health / Disease Incidence Data	21
1.5 Join Health Data to Spatial Data	23
1.6 Explore the Joined Dataset	24
1.7 Coordinate Reference System (CRS) Management	27
1.8 Practical Exercise	29
Module 1 Checklist	32
2 Module 2: Disease Mapping in R	33
2.1 Quick Thematic Maps with <code>mlspatial</code>	33
2.2 Add Country Labels to Maps	35
2.3 Create Disease Distribution Choropleth Maps with <code>ggplot2</code>	37
2.4 Create Interactive Maps with <code>tmap</code>	39
2.5 Overlay Multiple Layers	42
2.6 Map Layout and Export	45
2.7 Practical Exercise	48
Module 2 Checklist	51
3 Module 3: Spatial Autocorrelation and Cluster Analysis	52
3.1 Build Spatial Weights Matrix	53
3.2 Calculate Global Moran's I	54
3.3 Calculate Local Moran's I (LISA)	56
3.4 Hotspot Analysis (<code>Getis-Ord Gi*</code>)	58
3.5 Visualize Cluster Results	61
3.6 Practical Exercise	64

Module 3 Checklist	67
4 Module 4: Spatial Regression and Epidemiological Risk Factor Analysis	68
4.1 Simulate Risk-Factor Covariates	68
4.2 Build Ordinary Least Squares (OLS) Regression	71
4.3 Diagnosing Data Leakage	73
4.4 Refined Model	75
4.5 Test for Spatial Residual Autocorrelation	77
4.6 Geographically Weighted Regression (GWR)	78
4.7 Interpret and Visualize GWR Results	83
4.8 Practical Exercise	86
Module 4 Checklist	89
5 Module 5: Machine Learning for Spatial Epidemiology (Showcasing <code>mlspatial</code>)	90
5.1 Introduction to <code>mlspatial</code> for ML	90
5.2 Prepare Data for ML Modelling	91
5.3 Train a Random Forest Model	93
5.4 Train an XGBoost Model	96
5.5 Train a Support Vector Regression Model	99
5.6 Evaluate Model Performance	100
5.7 Honest Model Evaluation with <code>caret</code>	102
5.8 Identify Spatial Patterns and Risk Factors	105
5.9 Practical Exercise	112
Module 5 Checklist	115
6 Module 6: Integrated Application from Data to Decision	116
6.1 Overlay Disease Burden with Service Availability	116
6.2 Distance / Accessibility Analysis	119
6.3 Result Interpretation and Report Generation	121
6.4 Ethical Considerations	122
6.5 Practical Exercise	124
Closing Remarks	127

Welcome

This tutorial provides a complete, **R-only** learning pathway for spatial epidemiology. It is designed specifically for the beginners in spatial epidemiology. No prior spatial analysis experience is assumed, but you should be comfortable with basic R: installing packages, reading data, and using `dplyr` verbs such as `filter()`, `select()`, and `mutate()`.

i R-Only Focus

This tutorial is for the **Afredac beginner introduction to Spatial Epidemiology for Health Researchers** in collaboration with the **Ghana R Users Community**. **All tasks**—from data import and preparation to mapping, analysis, and modelling—are performed entirely in R.

What You Will Learn

We will work through six progressive modules:

Module	Title	Key Skills
1	R Environment Setup and Spatial Data Import	<code>sf</code> , <code>mlspatial</code> , CRS, joins
2	Disease Mapping in R	<code>tmap</code> , <code>ggplot2</code> , interactive maps, layout
3	Spatial Autocorrelation and Cluster Analysis	<code>spdep</code> , Moran's I, LISA, Gi* hotspots
4	Spatial Regression and Environmental Factor Analysis	<code>terra</code> , OLS, GWR, <code>Gwmodel</code>
5	Machine Learning for Spatial Epidemiology	<code>mlspatial</code> , RF, XGBoost, SVR
6	Integrated Application from Data to Decision	Accessibility, R Markdown, ethics

Prerequisites

- R (version 4.1)
- RStudio (recommended)
- Basic knowledge of `dplyr` and `ggplot2`
- No prior GIS or spatial analysis experience required

How to Use This Tutorial

1. Clone the [repo](#) from github and open the project file `Spatial Epidemiology R Tutorial.Rproj` in RStudio.
2. Work through each module in order (01 → 06).
3. Each module is a standalone `.qmd` file with runnable code chunks.
4. To render the entire book, run: `quarto::quarto_render()` in the R console.
5. To render a single module, open it and click **Render** or press `Ctrl+Shift+K`.

Key Reference

Azeez, A., & Noel, C. (2025). *Predictive Modelling and Spatial Distribution of Pancreatic Cancer in Africa Using Machine Learning-Based Spatial Model*. Zenodo. <https://doi.org/10.5281/zenodo.16529986>

Let's Begin!

Click **Next** or open [Module 1: R Environment Setup and Spatial Data Import](#) to start Module 1.

1 Module 1: R Environment Setup and Spatial Data Import

This module establishes your R spatial epidemiology toolkit and teaches you how to import, join, and inspect spatial and health data. By the end of this module, you will have a clean `sf` object ready for mapping and analysis.

What you will learn

- How to install and load the packages this tutorial relies on.
- What a *shapefile*, a *GeoPackage*, and a *raster* are, and how to import all three into R using `sf` and `terra`.
- Why the GeoPackage (`.gpkg`) format has largely replaced the legacy shapefile (`.shp`).
- How to preprocess (tidy) spatial data before analysis.
- How to join a table of health outcomes to spatial boundaries.
- How to inspect and manage the coordinate reference system (CRS).

A short note on *why* setup matters: most frustrating spatial errors — maps that look blank, points plotted in the wrong ocean, joins that silently drop countries — happen *before* any modelling. This module builds good habits that will save you hours later.

1.1 Install and Load R Packages

Purpose: Install and attach the core spatial epidemiology toolkit.

A **package** is a collection of R functions and data bundled together. Two steps are always involved:

1. **Install** (once per machine): `install.packages()` downloads the package from a repository (CRAN by default) and stores it on your computer.
2. **Load** (every session): `library()` attaches the package so its functions are available. You must re-load packages each time you start a new R session.

The table below shows the packages we will use throughout this tutorial and their roles:

Task	Package	Role
Vector spatial data	<code>sf</code>	Simple features standard for polygons, points, lines
Raster spatial data	<code>terra</code>	Fast raster extraction and processing
Thematic mapping	<code>tmap</code> , <code>ggplot2</code>	Static and interactive maps
Data manipulation	<code>dplyr</code> , <code>tidyr</code> , <code>readr</code>	Tidy data wrangling
Spatial statistics	<code>spdep</code>	Neighbours, Moran's I, Getis-Ord Gi*
Geographically weighted regression	<code>GWmodel</code>	Local regression modelling
Integrated ML + mapping	<code>mlspatial</code>	Import, map, model, evaluate
Machine learning engines	<code>randomForest</code> , <code>xgboost</code> , <code>e1071</code> , <code>caret</code>	RF, XGB, SVR algorithms

```
# Install any missing packages
pkgs <- c("sf", "terra", "tmap", "ggplot2", "dplyr", "tidyr", "readr",
          "spdep", "GWmodel", "mlspatial", "randomForest", "xgboost",
          "e1071", "caret", "knitr", "rmarkdown")

install_if_missing <- function(p) {
  if (!require(p, character.only = TRUE, quietly = TRUE)) {
    install.packages(p, repos = "https://cloud.r-project.org/")
    library(p, character.only = TRUE)
  }
}
invisible(sapply(pkgs, install_if_missing))
```

How this code works

The helper function avoids re-installing packages you already have:

- `pkgs <- c(...)` creates a *character vector* of package names.
- `install_if_missing <- function(p)` defines a function taking one package name `p`.
- `require(p, character.only = TRUE, quietly = TRUE)` tries to load the package. It returns `TRUE` if the package is available and `FALSE` if not — importantly, it does **not** stop with an error like `library()` would.
- `!require(...)` therefore means “the package is *not* available”. When that is true, we install it and then load it.

- `character.only = TRUE` tells `require()/library()` to treat `p` as a *value* (a string like `"sf"`) rather than as a literal name. Without it, R would look for a package literally called `p`.
- `sapply(pkgs, install_if_missing)` applies the function to every element of `pkgs` in turn (a one-line loop).
- `invisible()` hides the output of `sapply()`, keeping the console tidy.

You can run the same logic more manually for a single package:

```
install.packages("sf")
library(sf)
```

Speed Tip

If you are on a slow internet connection, install packages one at a time and restart RStudio after each batch of 3–4 packages.

Expected output: All packages load without error. `mlspatial` should be version 0.1.0.

1.2 Load Administrative Boundary Data

Purpose: Import polygon shapefiles representing administrative regions.

We demonstrate two approaches: (a) built-in data from `mlspatial` (fastest for learning), and (b) `sf::st_read()` for your own shapefiles. The second approach is expanded in the next section.

```
# Approach A: Built-in Africa shapefile from mlspatial
data(africa_shp, package = "mlspatial")

# Inspect the object
class(africa_shp)
```

```
[1] "sf"          "data.frame"
```

```
dim(africa_shp)
```

```
[1] 54  9
```

```
# Approach B: Import your own shapefile (commented out)
# ghana_regions <- sf::st_read("path/to/ghana_regions.shp")
```

i Understanding the `sf` object

`africa_shp` is an `sf` (simple features) object. It is essentially a normal **data frame** with one special extra column, `geometry`, that stores the spatial coordinates of each feature. This is why `class()` prints both `"sf"` and `"data.frame"`: you get all the familiar data-frame tools (filtering, summarising, joining) *plus* spatial tools.

- Each **row** is one feature (here, one country).
- Each **column** other than `geometry` is an *attribute* (e.g. country name, population).
- `dim()` returns 54 rows (54 African countries) and 9 columns, one of which is `geometry`.

Key parameters of `st_read()`: - `dsn` (data source name): the path to the file or folder containing the data — a local file, a directory, a zipped shapefile, or a URL. - `layer`: the layer name; needed whenever the source holds more than one layer (a GeoPackage, a File Geodatabase, or a folder of shapefiles). Supply it explicitly for `.gpkg` files. - `quiet = TRUE` suppresses progress messages.

Expected output: `africa_shp` is an `sf` data frame with a `geometry` column and attribute columns (including `NAME`).

1.3 Importing Your Own Spatial Data

Purpose: Read an external shapefile, GeoPackage, or raster into R and prepare (tidy) them for analysis. This is the skill you need once you move from the bundled example data to your own project.

Two fundamental data models

Spatial data comes in two flavours, and R uses a different package for each:

Model	What it stores	Package	Example
Vector	Discrete features as points, lines, or polygons (with attribute tables)	sf	Admin boundaries, health facilities, roads
Raster	A continuous grid of cells (pixels), each holding a value	terra	Elevation, temperature, land cover, rainfall

A **shapefile** is a *vector* format. Despite the `.shp` extension, it is actually several files that must travel together — `.shp` (geometry), `.dbf` (attribute table), `.shx` (spatial index), and `.prj` (the CRS definition). If you move only the `.shp` file, the data will not open.

A **GeoPackage** (`.gpkg`) is the modern container for the same kind of vector data: a *single* file, built on SQLite, that can hold several layers at once. It is covered in *Import a GeoPackage with sf* below.

A **raster** file (commonly `.tif/.tiff`, from “GeoTIFF”) is a grid of cells. Each cell covers a fixed area and holds a single value, which is why rasters are ideal for continuous surfaces such as elevation or climate.

1.3.1 Import a shapefile with sf

We use a small shapefile that ships with the `terra` package (the 12 cantons of Luxembourg). Replace the path with your own `.shp` file if you want to import your own data.

```
# Get the path to a built-in example shapefile
shp_path <- system.file("ex/lux.shp", package = "terra")
shp_path
```

```
[1] "C:/Users/gkagyen/AppData/Local/R/win-library/4.5/terra/ex/lux.shp"
```

```
# Read it into R as an sf object
lux <- sf::st_read(shp_path, quiet = TRUE)

# Quick look
class(lux)
```

```
[1] "sf"          "data.frame"
```

```
nrow(lux)
```

```
[1] 12
```

i How `st_read()` works

`st_read()` is the single most important function for vector data. Give it a file path and it returns an `sf` object, automatically reading the geometry, the attribute table, and the CRS from the accompanying files.

- `dsn` is the path (here passed positionally as `shp_path`).
- `quiet = TRUE` silences the layer/feature progress report.

1.3.2 Import a GeoPackage with `sf`

A **GeoPackage** (`.gpkg`) is an open standard (OGC) container built on SQLite. It stores one or more spatial layers in a *single* file, so there are no sidecar files to lose. The `sf` package ships real GeoPackages in its `gpkg/` folder, so there is nothing to download: we use the 100 North Carolina counties in `nc.gpkg`. Conveniently, the identical dataset also ships as a shapefile, so we can compare the two formats on the same data.

```
# Path to a GeoPackage that ships with sf (100 North Carolina counties)
gpkg_path <- system.file("gpkg/nc.gpkg", package = "sf")
basename(gpkg_path)
```

```
[1] "nc.gpkg"
```

```
# Inspect the container before reading: layers, geometry, features, CRS
sf::st_layers(gpkg_path)
```

Driver: GPKG

Available layers:

	layer_name	geometry_type	features	fields	crs_name
1	nc.gpkg	Multi Polygon	100	14	NAD27

```
# Read the layer into R as an sf object
nc_gpkg <- sf::st_read(gpkg_path, quiet = TRUE)

# The same dataset also ships as a shapefile...
nc_shp <- sf::st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

# ...so the two formats can be compared directly
class(nc_gpkg)
```

```
[1] "sf"          "data.frame"
```

```
dim(nc_gpkg)
```

```
[1] 100  15
```

```
sf::st_crs(nc_gpkg)$epsg
```

```
[1] 4267
```

```
# The geometry column keeps whatever name the file itself declares
names(nc_gpkg)[ncol(nc_gpkg)] # "geom"
```

```
[1] "geom"
```

```
names(nc_shp)[ncol(nc_shp)] # "geometry"
```

```
[1] "geometry"
```

```
# The attribute tables are identical
identical(
  sf::st_drop_geometry(nc_gpkg)[order(nc_gpkg$FIPS), ],
  sf::st_drop_geometry(nc_shp)[order(nc_shp$FIPS), ]
)
```

```
[1] TRUE
```

Either way `st_read()` returns an `sf` object, so the choice of format changes how the data is stored, not how you analyse it.

A GeoPackage can also hold layers with **no geometry at all** — it is a SQLite database first and a spatial format second.

```
# A geometry-free layer comes back as a plain data frame
nosp <- sf::st_read(system.file("gpkg/nospatial.gpkg", package = "sf"), quiet = TRUE)
class(nosp)
```

```
[1] "data.frame"
```

i How `st_layers()` and `st_read()` work with a GeoPackage

- `system.file("gpkg/nc.gpkg", package = "sf")` builds the path to a file inside the installed `sf` package. These examples live in `gpkg/`, not the more usual `extdata/`, so the subfolder must be named explicitly.
- `st_layers()` queries the container and reports every layer with its geometry type, feature count, field count, and CRS. Run it whenever you receive an unfamiliar `.gpkg` — the file name tells you nothing about how many layers are inside.
- `st_read(gpkg_path)` returns an `sf` object. When a GeoPackage holds more than one layer you must name the one you want with `layer = "..."`; otherwise `st_read()` silently returns the *first* layer, which may not be the one you intended.
- `st_drop_geometry()` strips the geometry and returns a plain data frame, which is why `identical()` can compare the two attribute tables directly. The `order()` calls align the rows by FIPS first, because the two files store them in a different order.
- The geometry column is named by the file rather than hard-coded: this GeoPackage layer calls it `geom`, whereas `st_read()` always names a shapefile's geometry column `geometry`. Both are ordinary `sf` objects.
- Nothing here is GeoPackage-specific: `st_read()` delegates to GDAL, so the same call reads `.gpkg`, `.shp`, `.geojson`, `.gdb`, and dozens of other formats. Only `dsn` changes.

i Why `.gpkg` is more modern than `.shp`

The ESRI Shapefile dates from the early 1990s and carries known design limitations. A GeoPackage is a single, open, relational container that avoids most of them:

Feature	.shp (shapefile)	.gpkg (GeoPackage)
Files on disk	A mandatory trio (.shp, .shx, .dbf) plus sidecars (.prj, .cpg, ...)	One file
Layers per file	One	Many
Field name length	Truncated to 10 characters	Up to 255 characters
Field types	Limited; no true dates, no 64-bit integers	Full SQLite type system
Geometry types	One type per file	Mixable, multiple geometry columns
Missing values	No true NULL; blank numerics often arrive as 0	NULL distinct from 0
File size	2 GB per component	No comparable practical ceiling
Text encoding	Ambiguous, needs a .cpg file	UTF-8 by definition
Keys and indexes	None	Primary keys, constraints, spatial indexes
Standard	De facto (ESRI)	Open (OGC / SQLite)

Two of these bite spatial epidemiology directly. The **10-character field name limit** silently truncates columns: none of the `nc` fields are long enough to lose characters, but join keys and outcome names in your own data may be, and a truncated key drops records without warning. And with **no true NULL**, a genuinely missing incidence rate may be stored and read back as 0, deflating rates and pulling maps toward zero. GeoPackage also keeps the CRS inside the file, so a stray `.prj` cannot be lost in transit, and one file is far easier to version, checksum, and share than five.

In practice: use `.gpkg` for anything you control, but expect to meet `.shp` constantly in legacy data and in exports from older GIS software. `st_read()` handles both, so the mental cost of switching is one file path.

💡 Where to find more GeoPackage examples

`sf` ships five more GeoPackages alongside `nc.gpkg`:

```
list.files(system.file("gpkg", package = "sf"))
```

They are handy for practice, and `b_pump.gpkg` is a small piece of history — a single point in Soho, London (longitude -0.1367, latitude 51.5133) marking the Broad Street water pump whose removal ended John Snow's 1854 cholera outbreak and helped establish epidemiology as a discipline.

When a data portal offers a choice, prefer `.gpkg`, then `.geojson`, then a zipped `.shp`. GeoJSON is also a single file and universally supported, but it is plain text, so it is much larger and slower than `.gpkg` for polygon layers. To convert a shapefile you already have, `sf::st_write(myshape, "out.gpkg")` is all it takes — no external GIS software required.

1.3.3 Inspect and tidy the vector data

Before analysis, check the object's geometry type, CRS, and validity, and tidy up anything that will cause trouble later.

```
# Geometry type and number of features
sf::st_geometry_type(lux, by_geometry = FALSE)
```

```
[1] POLYGON
18 Levels: GEOMETRY POINT LINESTRING POLYGON MULTIPOINT ... TRIANGLE
```

```
# Coordinate reference system (EPSG code)
sf::st_crs(lux)
```

Coordinate Reference System:

```
User input: WGS 84
wkt:
GEOGCRS["WGS 84",
  DATUM["World Geodetic System 1984",
    ELLIPSOID["WGS 84",6378137,298.257223563,
      LENGTHUNIT["metre",1]],
    PRIMEM["Greenwich",0,
      ANGLEUNIT["degree",0.0174532925199433]],
    CS[ellipsoidal,2],
    AXIS["latitude",north,
      ORDER[1],
      ANGLEUNIT["degree",0.0174532925199433]],
    AXIS["longitude",east,
      ORDER[2],
      ANGLEUNIT["degree",0.0174532925199433]],
  ID["EPSG",4326]]
```

```
# Are all geometries valid? (FALSE means some need fixing)
all(sf::st_is_valid(lux))
```

```
[1] TRUE
```

```
# Fix any invalid geometries
lux <- sf::st_make_valid(lux)

# Drop empty geometries (features with no coordinates)
lux <- lux[!sf::st_is_empty(lux), ]

# Tidy column names: lower-case, underscores instead of spaces
names(lux) <- gsub("[[:space:]]+", "_", tolower(names(lux)))
names(lux)
```

```
[1] "id_1"      "name_1"    "id_2"      "name_2"    "area"      "pop"       "geometry"
```

```
# Keep only the columns you need (canton name and population)
lux_sub <- lux[, c("name_2", "pop", "geometry")]
head(lux_sub)
```

Simple feature collection with 6 features and 2 fields

Geometry type: POLYGON

Dimension: XY

Bounding box: xmin: 5.74414 ymin: 49.69933 xmax: 6.528252 ymax: 50.18162

Geodetic CRS: WGS 84

	name_2	pop	geometry
1	Clervaux	18081	POLYGON ((6.02493 50.18162,...
2	Diekirch	32543	POLYGON ((6.177477 49.87787...
3	Redange	18664	POLYGON ((5.880909 49.8714,...
4	Vianden	5163	POLYGON ((6.129381 49.97283...
5	Wiltz	16735	POLYGON ((5.975572 50.02795...
6	Echternach	18899	POLYGON ((6.381573 49.84235...

i What each step does

- `st_geometry_type(..., by_geometry = FALSE)` reports the *single* shared geometry type (e.g. MULTIPOLYGON) rather than one per row.
- `st_crs()` shows the coordinate reference system (see the CRS section below).

- `st_is_valid()` checks each geometry for self-intersections and other defects; `st_make_valid()` repairs them. Invalid geometries are a common source of cryptic errors later.
- `st_is_empty()` flags features with no coordinates; we remove them.
- `gsub("[[:space:]]+", "_", tolower(names(lux)))` converts names like "NAME_2" to "name_2" and replaces spaces with underscores, so they are easier to type and safe to use in formulas.
- Subsetting with `lux[, c("name_2", "pop", "geometry")]` keeps only relevant columns (note `geometry` stays so it remains spatial).

1.3.4 Import a raster with terra

We use a small digital elevation model (DEM) that ships with **terra**. Elevation is a common covariate in spatial epidemiology — for example, altitude influences temperature and the distribution of vector-borne diseases.

```
# Path to a built-in example raster (a DEM)
r_path <- system.file("ex/elev.tif", package = "terra")
r_path
```

```
[1] "C:/Users/gkagyen/AppData/Local/R/win-library/4.5/terra/ex/elev.tif"
```

```
# Read it into R as a SpatRaster
elev <- terra::rast(r_path)

# Inspect it
elev
```

```
class       : SpatRaster
size        : 90, 95, 1  (nrow, ncol, nlyr)
resolution  : 0.008333333, 0.008333333  (x, y)
extent      : 5.741667, 6.533333, 49.44167, 50.19167  (xmin, xmax, ymin, ymax)
coord. ref. : lon/lat WGS 84 (EPSG:4326)
source      : elev.tif
name        : elevation
min value   :      141
max value   :      547
```

```
terra::nlyr(elev)      # number of layers (bands)
```

```
[1] 1
```

```
terra::res(elev)       # cell size in the x and y directions
```

```
[1] 0.008333333 0.008333333
```

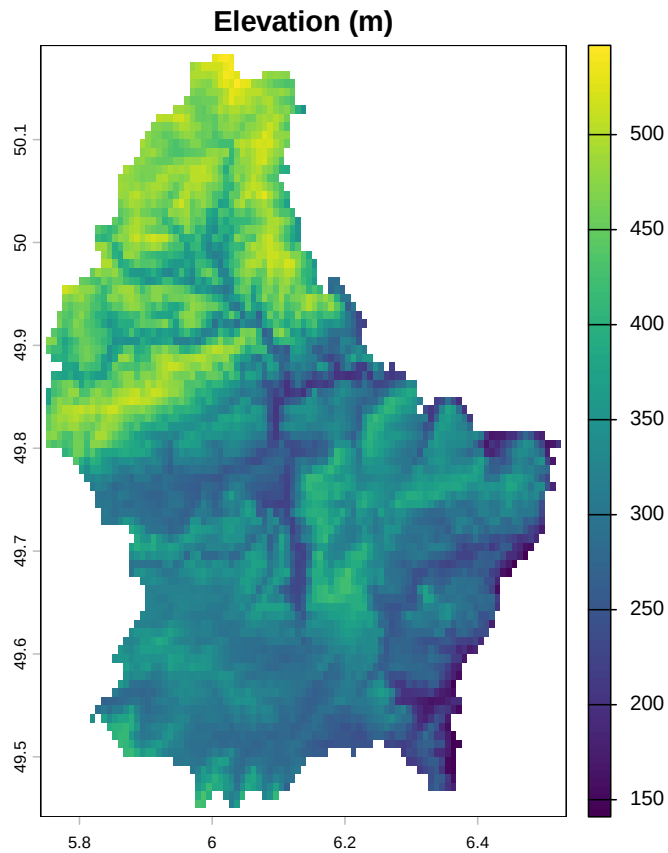
```
terra::ext(elev)       # spatial extent (bounding box)
```

```
SpatExtent : 5.741666666666667, 6.533333333333333, 49.441666666666667, 50.191666666666667 (xmin, xmax, ymin, ymax)
```

```
terra::crs(elev, describe = TRUE)$code # EPSG code
```

```
[1] "4326"
```

```
# Plot it  
plot(elev, main = "Elevation (m)")
```



i Understanding the raster print-out

`terra::rast()` returns a `SpatRaster`. When you print it, you see:

- **dimensions** — number of rows, columns, and layers. This example has one layer (a single-band DEM); a multispectral satellite image might have several.
- **resolution** — the ground size of one cell (here 0.0083 degrees).
- **extent** — the bounding box (xmin, xmax, ymin, ymax).
- **crs** — the coordinate reference system.

1.3.5 Preprocess the raster

Rasters from different sources rarely line up. The three most common preprocessing tasks are **reprojecting**, **cropping/masking**, and **resampling**.

```

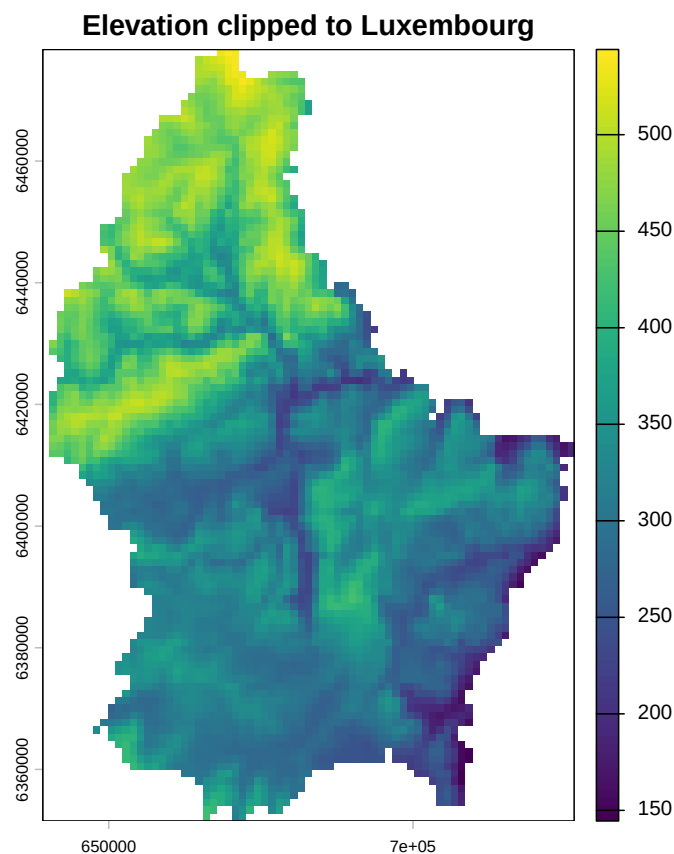
# Reproject the raster and polygons to a shared projected CRS (EPSG:3857)
elev_web <- terra::project(elev, "EPSG:3857")
lux_web <- sf::st_transform(lux_sub, crs = 3857)

# Convert the sf polygons to a SpatVector so terra can use them
lux_vect <- terra::vect(lux_web)

# Crop to the extent of the polygons, then mask to their exact shape
elev_lux <- terra::crop(elev_web, lux_vect)
elev_lux <- terra::mask(elev_lux, lux_vect)

plot(elev_lux, main = "Elevation clipped to Luxembourg")

```



Why these three steps matter

- `project()` changes the CRS of a raster; `st_transform()` does the same for vector data. Layers must share a CRS before you can overlay or stack them.
- `crop()` trims the raster to a bounding box (fast); `mask()` then sets cells *outside* the polygon to NA, giving the raster the exact shape of your study area.
- `vect()` converts an `sf` object into a `terra`-compatible `SpatVector` (and `sf::st_as_sf()` converts back the other way).

The `terra` package also provides `resample()` to align rasters that share a CRS but differ in resolution, which you will need when combining multiple raster layers.

The `mlspatial` shortcuts

`mlspatial` ships thin wrappers over these steps for convenience: `load_shapefile("path.shp")` (a wrapper around `st_read()`) and `load_incidence_data("path.xlsx")`. Because `load_shapefile()` passes the path straight through to `st_read()`, it reads a GeoPackage just as happily — `load_shapefile("path.gpkg")` returns the first layer. Under the hood these wrappers do exactly what we did above, so understanding the `sf/terra` functions first is important.

1.4 Load Health / Disease Incidence Data

Purpose: Import tabular disease data and link it to spatial boundaries.

```
# Built-in pancreatic cancer incidence data from mlspatial
data(panc_incidence, package = "mlspatial")

# Inspect structure
str(panc_incidence)
```

```
tibble [54 x 18] (S3: tbl_df/tbl/data.frame)
 $ NAME      : chr [1:54] "Algeria" "Angola" "Benin" "Botswana" ...
 $ incidence: num [1:54] 869 456 199 119 330 ...
 $ female   : num [1:54] 3569 1957 981 559 1683 ...
 $ male     : num [1:54] 5777 2940 1160 721 1869 ...
 $ ageb     : num [1:54] 1693 1343 439 296 670 ...
 $ agec     : num [1:54] 7642 3547 1699 982 2878 ...
```

```

$ agea      : num [1:54] 10.53 7.17 3.12 1.01 4.6 ...
$ fageb     : num [1:54] 660.4 460.7 150.2 91.7 250.3 ...
$ fagec     : num [1:54] 2902 1492 828 466 1429 ...
$ fagea     : num [1:54] 6.45 4.242 2.223 0.557 3.413 ...
$ mageb     : num [1:54] 1033 882 289 204 419 ...
$ magec     : num [1:54] 4740 2055 870 516 1448 ...
$ magea     : num [1:54] 4.082 2.931 0.892 0.456 1.184 ...
$ yra       : num [1:54] 468.3 239.9 107.8 56.2 182.4 ...
$ yrb       : num [1:54] 496 254.6 111.2 57.9 187.2 ...
$ yrc       : num [1:54] 523.8 270.3 117.1 60.4 193.1 ...
$ yrd       : num [1:54] 545.7 286.8 124.6 63.1 198.5 ...
$ yre       : num [1:54] 563.8 299.6 129.5 64.5 204.3 ...

```

```
head(panc_incidence)
```

```

# A tibble: 6 x 18
  NAME      incidence female  male  ageb  agec  agea fageb fagec fagea mageb magec
  <chr>      <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Algeria      869.   3569. 5777. 1693. 7642. 10.5  660.   2902.  6.45  1033. 4740.
2 Angola       456.   1957. 2940. 1343. 3547.  7.17 461.   1492.  4.24   882. 2055.
3 Benin        199.    981. 1160.  439. 1699.  3.12 150.    828.  2.22   289.  870.
4 Botswa~     119.    559.  721.  296.  982.  1.01 91.7   466.  0.557  204.  516.
5 Burkin~     330.   1683. 1869.  670. 2878.  4.60 250.   1429.  3.41   419. 1448.
6 Burundi     163.    837.  915.  420. 1326.  6.23 174.    658.  4.88   246.  667.
# i 6 more variables: magea <dbl>, yra <dbl>, yrb <dbl>, yrc <dbl>, yrd <dbl>,
#   yre <dbl>

```

i Reading `str()` and `head()`

- `str()` (structure) lists each column, its type, and the first few values. Use it to confirm that columns you expect to be numeric really are numeric, and to spot factor/character mix-ups.
- `head()` shows the first six rows so you can sanity-check the values.

Key variables in `panc_incidence`: - `NAME`: Country name (the join key) - `incidence`: Incidence rate per 100,000 population (as documented in `mlspatial`) - `female`, `male`: Counts by sex - `agea`, `ageb`, `agec`: Age-stratum values (below 20, 20–54, and above 55 years) - `yra` ... `yre`: Yearly incidence rates (2017–2021)

Expected output: A tibble with one row per country (54 rows) and 18 columns.

1.5 Join Health Data to Spatial Data

Purpose: Attach disease attributes to the correct spatial polygons so we can map and model them.

A join matches rows from two tables using a common **key** column. Here the key is the country name. Crucially, the spelling must match exactly — in our data the shapefile spells the country “Côte d’Ivoire” (with an accent), while the health table spells it “Cote d’Ivoire” (without). Without fixing this, the join would silently drop the country. We fix the name first.

```
# Fix the one mismatched name so the join keys align
africa_shp <- africa_shp |>
  mutate(
    NAME      = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME),
    COUNTRYAFF = gsub("Côte d'Ivoire", "Cote d'Ivoire", COUNTRYAFF)
  )

# Method 1: mlspatial wrapper (recommended for consistency)
africa_health <- mlspatial::join_data(
  sf_data = africa_shp,
  tbl_data = panc_incidence,
  by = "NAME"
)

# Method 2: dplyr inner_join (identical result, more familiar syntax)
# africa_health <- africa_shp |>
#   dplyr::inner_join(panc_incidence, by = "NAME")

# Verify the join
dim(africa_health)
```

```
[1] 54 26
```

How the code works

- `gsub(pattern, replacement, x)` performs a **global substitution**: it replaces every occurrence of "Côte d'Ivoire" with "Cote d'Ivoire" in the NAME column. We do the same for COUNTRYAFF so every name column is consistent.
- `mutate()` (from `dplyr`) creates or overwrites columns.
- `join_data()` is an `mlspatial` convenience wrapper; inspecting its source shows it simply calls `dplyr::inner_join()`.
- `by = "NAME"` tells R which column is the shared key. Both tables **must** use identical

spelling and casing.

- `inner_join()` keeps only rows that match in both tables. Use `left_join()` if you want to keep all polygons even those without health data (unmatched rows get `NA`).

Expected output: An `sf` object with the same number of rows as the original shapefile (54, because every country matched) and all columns from both datasets.

1.6 Explore the Joined Dataset

Purpose: Confirm that the join worked and understand the data structure.

```
# Structure of the sf object
str(africa_health)
```

```
Classes 'sf' and 'data.frame':  54 obs. of  26 variables:
```

```
$ OBJECTID   : int   2 3 4 5 6 7 8 9 10 11 ...
$ FIPS_CNTRY : chr   "UV" "CV" "IV" "GA" ...
$ ISO_2DIGIT : chr   "BF" "CV" "CI" "GM" ...
$ ISO_3DIGIT : chr   "BFA" "CPV" "CIV" "GMB" ...
$ NAME       : chr   "Burkina Faso" "Cabo Verde" "Cote d'Ivoire" "Gambia" ...
$ COUNTRYAFF : chr   "Burkina Faso" "Cabo Verde" "Cote d'Ivoire" "Gambia" ...
$ CONTINENT  : chr   "Africa" "Africa" "Africa" "Africa" ...
$ TOTPOP     : int  20107509 560899 24184810 2051363 27499924 12413867 1792338 4689021 17885...
$ incidence  : num   330.4 53.4 320.4 31.4 856.3 ...
$ female     : num   1683 362 1592 140 4566 ...
$ male       : num   1869 211 1852 197 4640 ...
$ ageb       : num   669.7 93.7 856.6 68.7 2047 ...
$ agec       : num   2878 480 2579 268 7147 ...
$ agea       : num    4.597 0.265 8.744 0.718 11.888 ...
$ fageb      : num   250.3 40.2 322.8 23.1 782 ...
$ fagec      : num   1429 322 1262 116 3775 ...
$ fagea      : num    3.413 0.146 7.292 0.548 8.816 ...
$ mageb      : num   419.5 53.5 533.8 45.6 1265 ...
$ magec      : num   1448 158 1317 152 3372 ...
$ magea      : num    1.18 0.12 1.45 0.17 3.07 ...
$ yra        : num   182.4 30.2 158.6 16.6 524.7 ...
$ yrb        : num   187.2 34.1 164.1 17.1 552.6 ...
$ yrc        : num    193 35 168 18 578 ...
```



```

3 158.59941 164.13910 168.28467 175.80819 182.24539
4 16.60058 17.06432 18.00591 18.29294 18.67213
5 524.74512 552.56356 578.46814 602.67088 621.47540
6 73.10342 74.94067 76.85661 78.59267 79.42589

```

```

      geometry
1 MULTIPOLYGON (((102188.3 12...
2 MULTIPOLYGON (((-2618884 16...
3 MULTIPOLYGON (((-594413.7 5...
4 MULTIPOLYGON (((-1848987 15...
5 MULTIPOLYGON (((-352664.2 6...
6 MULTIPOLYGON (((-884637.8 8...

```

```

# Summary of the incidence variable
summary(africa_health$incidence)

```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.765	112.148	185.325	528.383	687.312	4950.860

```

# Check for missing values in key variables
sum(is.na(africa_health$incidence))

```

```
[1] 0
```

i Reading the output

- `str()` should show a Classes 'sf' ... object with a `geometry` column of type MULTIPOLYGON.
- `summary()` gives the five-number summary plus the mean for a numeric column. `incidence` is numeric and ranges from about 1.8 to 4950.9, with a median near 185.
- `sum(is.na(...))` counts missing values. 0 means every country has a recorded incidence, so no imputation is needed here.

Expected output: - `str()` shows a Classes 'sf' ... object with a `geometry` column of type MULTIPOLYGON. - `incidence` is numeric with a range of roughly 1.8 to 4900 and no missing values.

1.7 Coordinate Reference System (CRS) Management

Purpose: Ensure all spatial layers share the same projection. Mismatched CRS is the most common source of silent errors in spatial analysis.

A **coordinate reference system** defines how the flat coordinates in your data relate to locations on the Earth. It has two parts: a **datum** (a model of the Earth's shape) and a **projection** (how the curved Earth is flattened onto a map). CRSs are identified by **EPSG codes** (e.g. 4326, 3857).

```
# View current CRS
sf::st_crs(africa_health)
```

Coordinate Reference System:

User input: WGS 84 / Pseudo-Mercator

wkt:

```
PROJCRS["WGS 84 / Pseudo-Mercator",
  BASEGEOGCRS["WGS 84",
    ENSEMBLE["World Geodetic System 1984 ensemble",
      MEMBER["World Geodetic System 1984 (Transit)"],
      MEMBER["World Geodetic System 1984 (G730)"],
      MEMBER["World Geodetic System 1984 (G873)"],
      MEMBER["World Geodetic System 1984 (G1150)"],
      MEMBER["World Geodetic System 1984 (G1674)"],
      MEMBER["World Geodetic System 1984 (G1762)"],
      MEMBER["World Geodetic System 1984 (G2139)"],
      MEMBER["World Geodetic System 1984 (G2296)"],
      ELLIPSOID["WGS 84",6378137,298.257223563,
        LENGTHUNIT["metre",1]],
      ENSEMBLEACCURACY[2.0]],
    PRIMEM["Greenwich",0,
      ANGLEUNIT["degree",0.0174532925199433]],
    ID["EPSG",4326]],
  CONVERSION["Popular Visualisation Pseudo-Mercator",
    METHOD["Popular Visualisation Pseudo Mercator",
      ID["EPSG",1024]],
    PARAMETER["Latitude of natural origin",0,
      ANGLEUNIT["degree",0.0174532925199433],
      ID["EPSG",8801]],
    PARAMETER["Longitude of natural origin",0,
      ANGLEUNIT["degree",0.0174532925199433],
      ID["EPSG",8802]],
```

```

    PARAMETER["False easting",0,
      LENGTHUNIT["metre",1],
      ID["EPSG",8806]],
    PARAMETER["False northing",0,
      LENGTHUNIT["metre",1],
      ID["EPSG",8807]]],
  CS[Cartesian,2],
  AXIS["easting (X)",east,
    ORDER[1],
    LENGTHUNIT["metre",1]],
  AXIS["northing (Y)",north,
    ORDER[2],
    LENGTHUNIT["metre",1]],
  USAGE[
    SCOPE["Web mapping and visualisation."],
    AREA["World between 85.06°S and 85.06°N."],
    BBOX[-85.06,-180,85.06,180]],
  ID["EPSG",3857]]

```

```

# Check if it is geographic (degrees) or projected (meters)
sf::st_is_longlat(africa_health)

```

```
[1] FALSE
```

```

# If you need to transform to a projected CRS for distance calculations
# Example: Africa Albers Equal Area Conic (EPSG: 102022)
# africa_projected <- sf::st_transform(africa_health, crs = 102022)

```

i Geographic vs projected CRS

- **Geographic** CRSs (e.g. EPSG:4326, WGS 84) use degrees of latitude/longitude. `st_is_longlat()` returns TRUE for these. Degrees are *not* a consistent unit of distance (a degree of longitude shrinks toward the poles), so they are unsuitable for measuring distances or areas.
- **Projected** CRSs (e.g. EPSG:3857, or UTM zones) use metres on a flat surface. Distances and areas are meaningful, but every projection distorts *something* (shape, area, or distance).
- `st_transform()` converts between CRSs. It changes the stored coordinates but not the underlying data.

Key concepts: - **EPSG:3857** (WGS 84 / Pseudo-Mercator): a web mapping projection in metres, used by Google Maps and OpenStreetMap. It is convenient for visualisation but distorts

areas and distances, which worsens away from the equator. - **Projected CRS** (e.g. UTM zones, Albers Equal Area): distances in metres, with distortion controlled for a specific region. Essential for buffers, distances, and GWR bandwidth selection.

For this tutorial we keep EPSG:3857 because `tmap` handles it well for visualisation, but we note that distance calculations may be distorted.

Expected output: `st_crs()` returns the CRS definition (EPSG:3857, WGS 84 / Pseudo-Mercator). `st_is_longlat()` returns `FALSE` because this is a projected CRS with units in metres, not geographic degrees.

1.8 Practical Exercise

Now apply what you have learned. Work through the tasks below, then check your answers against the solution.

1. Load `africa_shp` and report its class, dimensions, geometry type, and CRS code.
2. Import the bundled Luxembourg shapefile (`system.file("ex/lux.shp", package = "terra")`). How many cantons does it contain, and what geometry type does it use?
3. Import the bundled elevation raster (`system.file("ex/elev.tif", package = "terra")`) and report its resolution, extent, and CRS code.
4. Re-project that raster (in Q3) to EPSG:3857 and confirm the new CRS.
5. Recreate the country join yourself using `inner_join()` (without `mlspatial::join_data()`), remembering to fix the “Côte d’Ivoire” spelling first. How many rows does the result have?
6. Read the GeoPackage that ships with `sf` (`system.file("gpkg/nc.gpkg", package = "sf")`) and use `st_layers()` to report its layer name, feature count, and geometry type. What is that layer’s geometry column called? Then read the equivalent shapefile (`system.file("shape/nc.shp", package = "sf")`) and explain why both objects behave identically.
7. (*Challenge*) Use `st_is_valid()` on `africa_shp` and report how many countries (if any) have invalid geometries. If any are invalid, repair them with `st_make_valid()`.

i Model solution

```

library(sf)
library(terra)
library(dplyr)
library(mlspatial)

# 1. Built-in Africa shapefile
data(africa_shp, package = "mlspatial")
class(africa_shp) # "sf" "data.frame"
dim(africa_shp) # 54 rows x 9 columns
sf::st_geometry_type(africa_shp, by_geometry = FALSE) # MULTIPOLYGON
sf::st_crs(africa_shp)$epsg # 3857

# 2. Bundled Luxembourg shapefile
lux <- sf::st_read(system.file("ex/lux.shp", package = "terra"), quiet = TRUE)
nrow(lux) # 12 cantons
sf::st_geometry_type(lux, by_geometry = FALSE) # POLYGON

# 3. Bundled elevation raster
elev <- terra::rast(system.file("ex/elev.tif", package = "terra"))
terra::res(elev) # 0.008333333 0.008333333
terra::ext(elev) # 5.741667 6.533333 49.44167 50.19167
terra::crs(elev, describe = TRUE)$code # 4326

# 4. Reproject
elev_web <- terra::project(elev, "EPSG:3857")
terra::crs(elev_web, describe = TRUE)$code # 3857

# 5. Manual join
data(panc_incidence, package = "mlspatial")
africa_fixed <- africa_shp |>
  mutate(NAME = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME))
africa_health2 <- dplyr::inner_join(africa_fixed, panc_incidence, by = "NAME")
nrow(africa_health2) # 54

# 6. GeoPackage shipped with sf
gpkg_path <- system.file("gpkg/nc.gpkg", package = "sf")
sf::st_layers(gpkg_path) # layer "nc.gpkg", 100 features, MULTIPOLYGON
nc_gpkg <- sf::st_read(gpkg_path, quiet = TRUE)
names(nc_gpkg)[ncol(nc_gpkg)] # "geom"

nc_shp <- sf::st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
nrow(nc_shp) # 100 - same data, geometry column "geometry"

# 7. Validity check
sum(!sf::st_is_valid(africa_shp)) # number of invalid geometries
africa_shp <- sf::st_make_valid(africa_shp)

```

Module 1 Checklist

- ☐ All packages installed and loaded
- ☐ `africa_shp` imported successfully
- ☐ You can import an external shapefile with `st_read()`
- ☐ You can read a GeoPackage and explain why `.gpkg` supersedes `.shp`
- ☐ You can import and preprocess an external raster with `terra`
- ☐ `panc_incidence` loaded and inspected
- ☐ `africa_health` created via `join_data()`
- ☐ CRS verified and understood
- ☐ Practical exercise completed

Next: Proceed to [Module 2: Disease Mapping in R](#).

2 Module 2: Disease Mapping in R

This module teaches you to create publication-quality thematic maps using `mlspatial`, `tmap`, and `ggplot2`. You will learn static maps, interactive maps, multi-layer overlays, and export techniques.

What you will learn

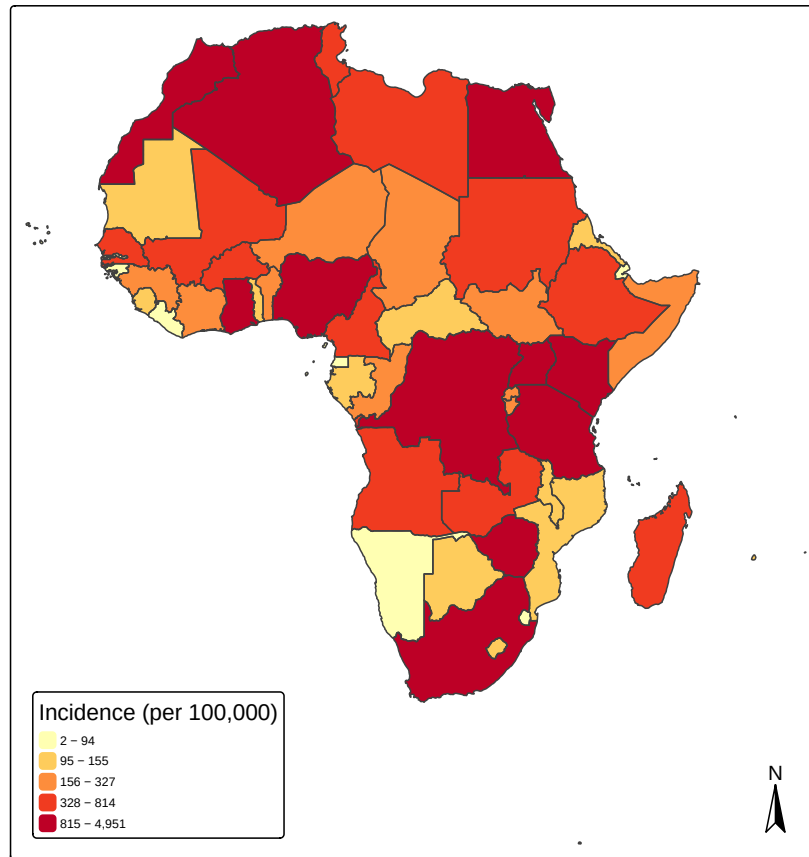
- How to build a choropleth (thematic) map in a single line with `mlspatial`.
- How `tmap` assembles a map from stacked *layers*.
- How to reproduce and customise the same maps with `ggplot2`.
- How to switch between static and interactive (Leaflet) maps.
- How to overlay point data on polygons and export multi-panel figures.

A **choropleth** map colours each region according to a data value (here, disease incidence). It is the single most common way to visualise spatial variation in health outcomes, so this module focuses on getting the colour scale, legend, and labels right.

2.1 Quick Thematic Maps with `mlspatial`

Purpose: Generate a publication-ready choropleth map in one line.

```
map1 <- mlspatial::plot_single_map(  
  sf_data = africa_health,  
  var = "incidence",  
  title = "Incidence (per 100,000)",  
  palette = "yl_or_rd"  
)  
map1
```



i How `plot_single_map()` works

This convenience function hides several `tmap` steps behind one call:

- `sf_data`: the `sf` object to draw.
- `var`: the column to colour by — **quoted**, because it is passed as a string to the plotting engine.
- `title`: the legend title.
- `palette`: the colour scheme. It accepts any ColourBrewer or viridis name written in lower-case with underscores (e.g. "yl_or_rd", "blues").

Under the hood, `plot_single_map()` calls `tm_fill()` with a *quantile* scale. That means it divides the incidence values into five bins, each containing an equal number of countries, and colours each bin. This is robust to skewed data (a few very high values would otherwise compress all other colours into one bin).

Key parameters: - `var`: Column name to map (must be quoted). - `palette`: Any RColour-

Brewer or viridis palette name. "yl_or_rd" (old name "YlOrRd") is standard for public-health “hot” maps.

Expected output: A static tmap object showing African countries coloured by pancreatic cancer incidence.

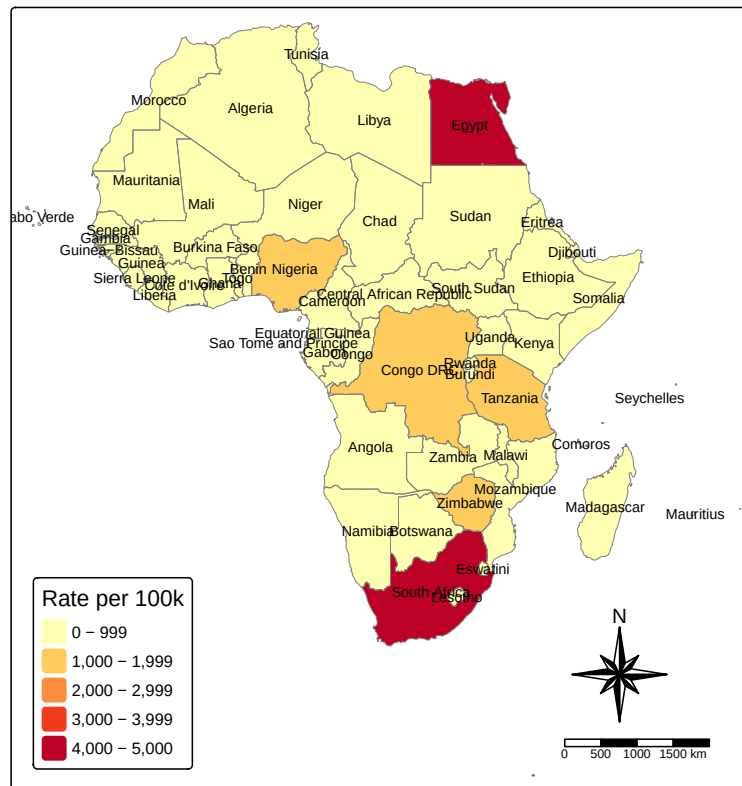
2.2 Add Country Labels to Maps

Purpose: Annotate maps so stakeholders can identify regions without a separate legend.

```
labelled_map <- tm_shape(africa_health) +
  # Fill layer (colours the polygons)
  tm_fill(
    "incidence",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Rate per 100k")
  ) +
  # Border layer (outlines)
  tm_borders(col = "grey50", lwd = 0.6) +
  # Text layer (adds country names)
  tm_text(
    "NAME",
    size = 0.6,
    col = "black"
  ) +
  # Title layer
  tm_title("Pancreatic Cancer Incidence in Africa",
    color = 'black', size = 1.2) +
  # Layout (everything else about map appearance)
  tm_layout(
    legend.position = c("left", "bottom")
  ) +
  # Add compass symbol
  tm_compass(
    type = "8star",
    position = c("right", "bottom")
  ) +
  # Add scale bar
  tm_scalebar(
```

```
position = c("right", "bottom")
)
labelled_map
```

Pancreatic Cancer Incidence in Africa



```
# You can also create a quick map using the qtm() function (less flexible)
# qtm(africa_health, fill = "incidence", fill.scale = tm_scale(values = "brewer.yl_or_rd"))
```

i The `tmap` layer grammar

`tmap` builds maps the way `ggplot2` builds plots: by **adding layers** with `+`. Each `tm_*()` function adds one element:

- `tm_shape(data)` declares the data for the layers that follow it. It does not draw anything by itself.
- `tm_fill("incidence")` colours each polygon according to `incidence`.
- `tm_borders()` draws the polygon outlines.

- `tm_text("NAME")` places the value of the NAME column at each polygon's centroid.
- `tm_title()`, `tm_compass()`, `tm_scalebar()`, and `tm_layout()` add map furniture (title, north arrow, scale bar, and general layout).

In `tmap` version 4, styling is split into two arguments: `*.scale` controls **how values map to visuals** (colour palette, breaks), and `*.legend` controls **how the legend looks** (title, position). So `fill.scale = tm_scale(values = "brewer.yl_or_rd")` sets the palette, while `fill.legend = tm_legend(title = "Rate per 100k")` sets the legend title.

Key parameters: - `tm_text()`: `text = "NAME"` specifies the label column. - `size` is in map units; adjust downward for small countries.

Expected output: A choropleth map with country names overlaid at centroids.

2.3 Create Disease Distribution Choropleth Maps with ggplot2

Purpose: Leverage the tidyverse grammar of graphics for highly customized static maps.

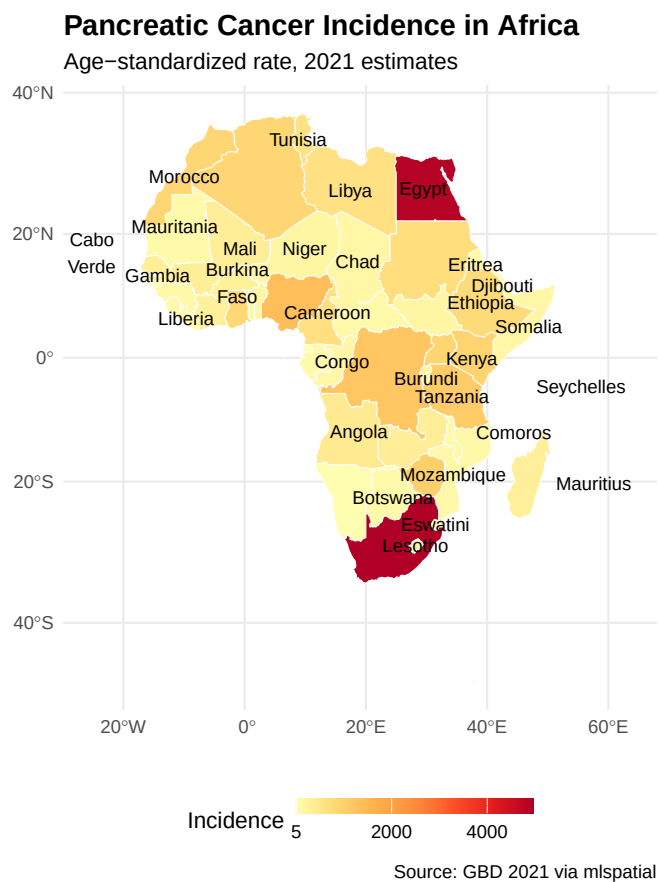
```
library(ggplot2)

ggplot(africa_health) +
  geom_sf(aes(fill = incidence), colour = "white", linewidth = 0.2) +
  scale_fill_distiller(
    palette = "YlOrRd", direction = 1,
    name = "Incidence",
    breaks = c(5, 2000, 4000)
  ) +
  theme_minimal() +
  labs(
    title = "Pancreatic Cancer Incidence in Africa",
    subtitle = "Age-standardized rate, 2021 estimates",
    caption = "Source: GBD 2021 via mlspatial",
    x = "", y = ""
  ) +
  # geom_text(
  #   aes(label = stringr::str_wrap(NAME, 8), geometry = geometry),
  #   stat = "sf_coordinates",
  #   colour = 'black', size = 3
  # ) +
```

```

geom_sf_text(
  aes(label = stringr::str_wrap(NAME, 8)),
  size = 3.2, colour = 'black',
  check_overlap = TRUE # avoids overlapping labels
) +
theme(
  legend.position = "bottom",
  plot.title = element_text(face = "bold", size = 14)
) +
guides(
  fill = guide_colourbar(barheight = 0.5, barwidth = 8)
)

```



i The grammar of graphics, mapped

`ggplot2` builds a plot from three ideas — **data**, **aesthetics**, and **geometries** — layered with `+`:

- `ggplot(africa_health)` supplies the data.
- `geom_sf()` is the geometry for spatial data. Inside `aes(fill = incidence)` we *map* the `incidence` column to the fill colour: each country gets a colour according to its value.
- `colour = "white"` and `linewidth = 0.2` are set **outside** `aes()`, meaning they are fixed values (thin white borders) rather than mapped to data.
- `scale_fill_distiller()` controls the fill *scale*: the `YlOrRd` palette, `direction = 1` (light-to-dark), the legend `name`, and `breaks` (where tick labels appear on the legend).
- `geom_sf_text()` adds labels. `stringr::str_wrap(NAME, 8)` inserts line breaks so long names fit, and `check_overlap = TRUE` drops labels that would collide.
- `theme()` and `guides()` fine-tune appearance (legend position and title size).

Key parameters: - `geom_sf()`: The workhorse for sf objects. `aes(fill = incidence)` maps the variable to colour. - `scale_fill_distiller()`: For continuous data. Perceptually uniform, colour-blind safe, and prints well in greyscale.

Expected output: A static ggplot map with a continuous colour legend.

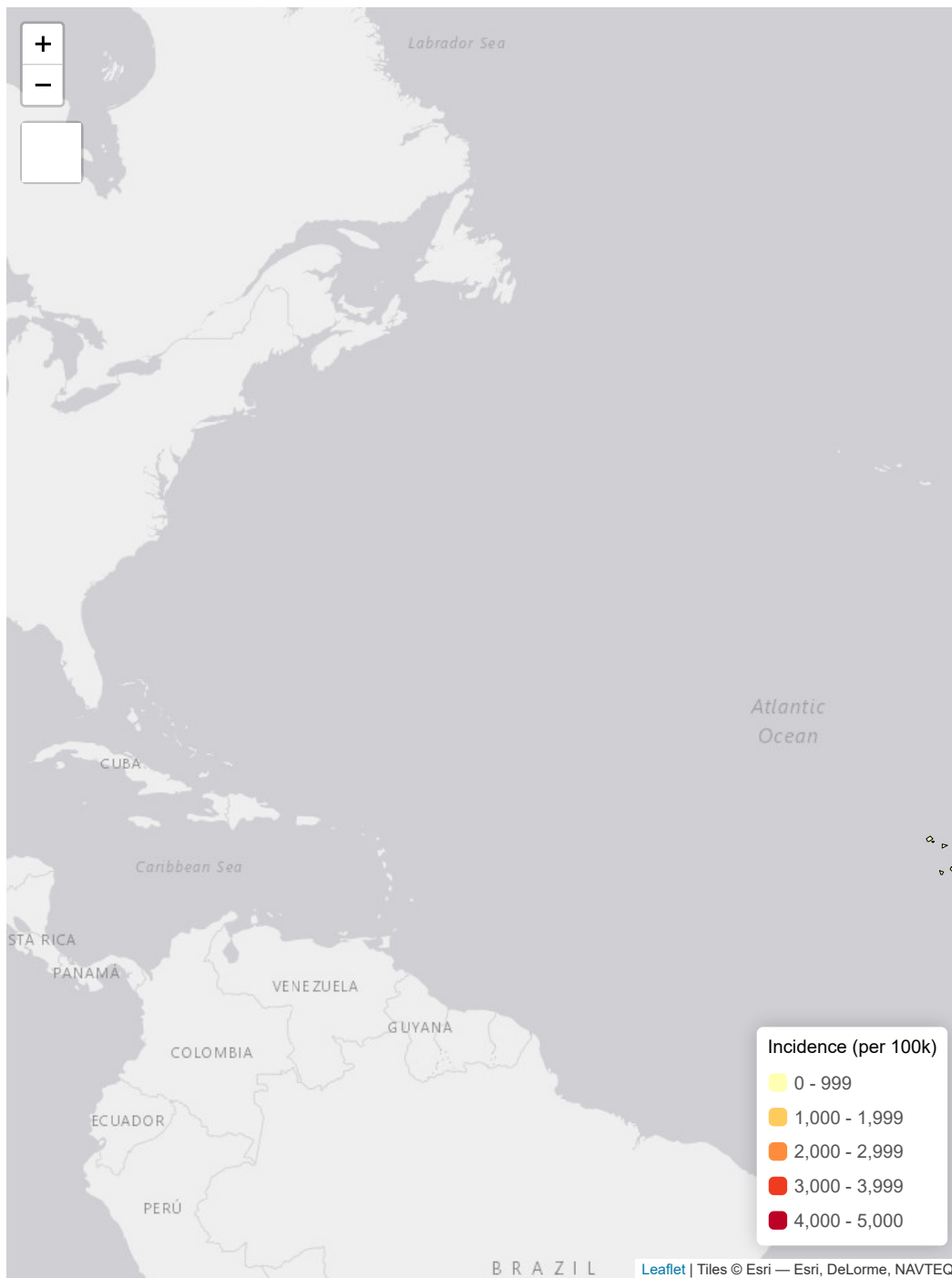
2.4 Create Interactive Maps with `tmap`

Purpose: Allow panning, zooming, and layer toggling for exploratory data analysis.

```
# Switch tmap to interactive mode
tmap_mode("view")

interactive_map <- tm_shape(africa_health) +
  tm_fill(
    "incidence",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Incidence (per 100k)",
    popup.vars = c("incidence", "female", "male")
  ) +
  tm_borders(col = "black", lwd = 0.5)
```

```
interactive_map
```



```
# Return to static mode for publication figures
tmap_mode("plot")
```

i Static vs interactive

tmap has two rendering modes controlled by `tmap_mode()`:

- "plot" (default) draws a static image, ideal for documents and publication.
- "view" renders the same map as a **Leaflet** JavaScript widget that you can pan, zoom, and click.

The map code itself is identical in both modes — you only change the mode. `popup.vars` lists the columns shown in a pop-up when a user clicks a polygon, which is only relevant in interactive mode. The chunk is marked `eval: false` so the HTML widget is not embedded in the book; run it in RStudio to interact with it.

Key parameters: - `tmap_mode("view")`: Renders to a Leaflet JavaScript widget. - `popup.vars`: Vector of column names shown when a user clicks a polygon.

Expected output: An interactive HTML widget (in RStudio Viewer or browser) of the Africa map that you can zoom and pan with clickable pop-ups.

2.5 Overlay Multiple Layers

Purpose: Combine disease polygons with point locations (e.g., cases, facilities) to reveal spatial relationships.

```
set.seed(4607)

# Simulate 70 health facility points across Africa for demonstration
facility_points <- st_sample(africa_health, size = 70, type = "random") |>
  st_as_sf() |>
  mutate(facility_id = paste0("FAC_", row_number()),
         type = "Health Facilities")

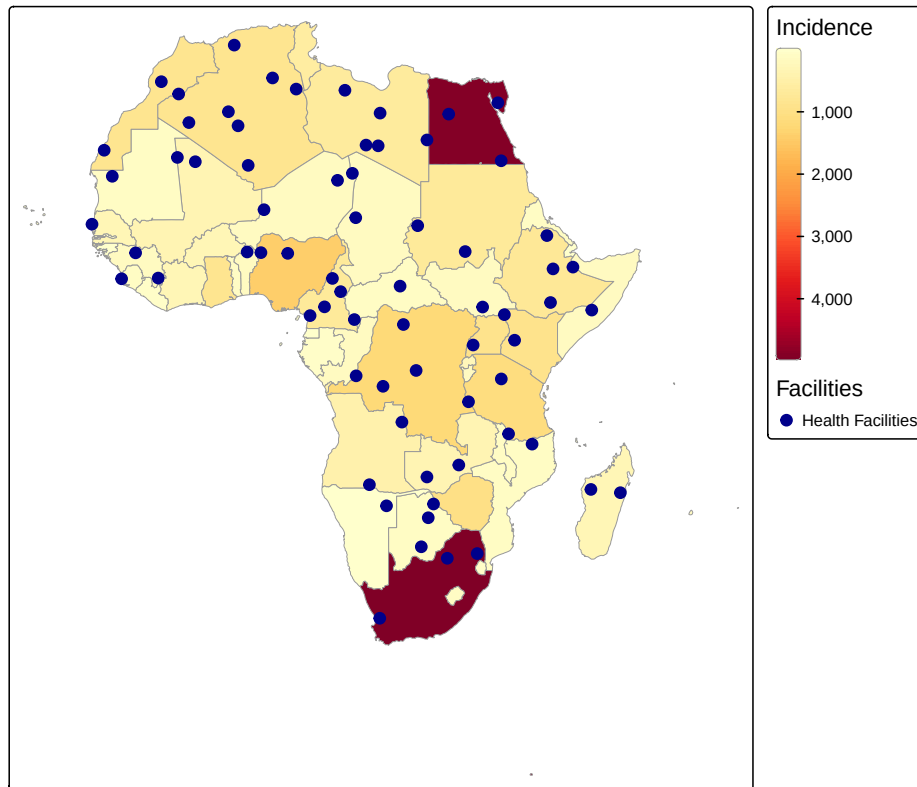
# Multi-layer map
overlay_map <- tm_shape(africa_health) +
  tm_polygons(
    "incidence",
```

```

    fill.scale = tm_scale_continuous(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Incidence",
                            orientation = 'portrait'),
    col = "grey60", lwd = 0.5) +
tm_shape(facility_points) +
tm_dots(
  fill = "type",
  fill.scale = tm_scale_categorical(values = "darkblue"),
  fill.legend = tm_legend(title = "Facilities"),
  size = 0.5
) +
# tm_add_legend(
#   type = "symbols",
#   labels = "Health Facilities",
#   fill = "darkblue",
#   shape = 16, # Standard round solid dot
#   size = 0.5, # Slightly scaled up visually just for the legend box
#   title = "Facilities"
# ) +
tm_title(
  "Disease Burden & Health Facility Locations", size = 1.2
) +
tm_layout(
  component.autoscale = FALSE
)
overlay_map

```

Disease Burden & Health Facility Locations



i Simulating points and stacking layers

- `set.seed(4607)` makes the random points reproducible — re-running the chunk gives the same 70 points every time.
- `st_sample(africa_health, size = 70, type = "random")` places 70 random points *inside* the polygons. `st_as_sf()` wraps them as an `sf` object, and `mutate()` adds an ID column (`paste0("FAC_", row_number())`) and a category label.
- The map then calls `tm_shape()` **twice**: first for the polygons, then for the points. Each `tm_shape()` switches the active data, so the layers that follow it draw that data. `tm_polygons()` fills and outlines the countries; `tm_dots()` draws the points as filled circles.
- `tm_scale_categorical(values = "darkblue")` is used because `type` is a categorical variable (all points are “Health Facilities”), so a single colour — not a ramp — is appropriate.

Key parameters: - `tm_shape()` can be called multiple times; each subsequent layer refers to the most recent shape. - `tm_dots()` controls point symbology.

Expected output: A single map showing incidence fills overlaid with blue facility dots.

2.6 Map Layout and Export

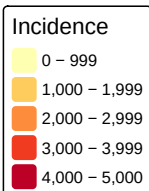
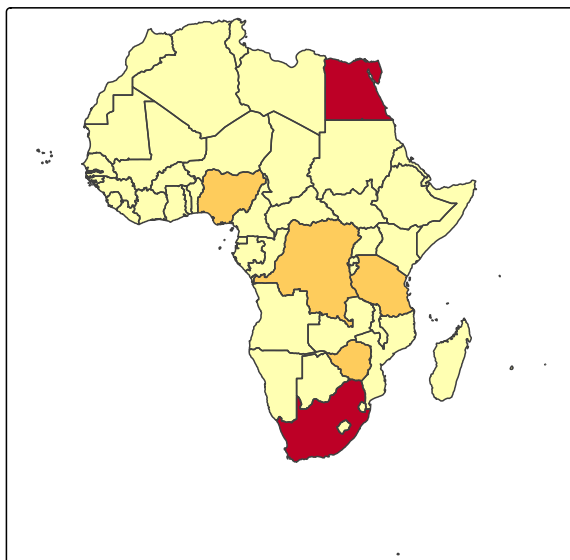
Purpose: Arrange multiple maps side-by-side and save publication-quality figures.

```
# Create two thematic maps
# Map 1
map_inc <- tm_shape(africa_health) +
  tm_fill(
    "incidence",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Incidence")
  ) +
  tm_borders() +
  tm_title("A. Incidence")

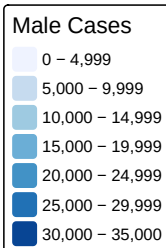
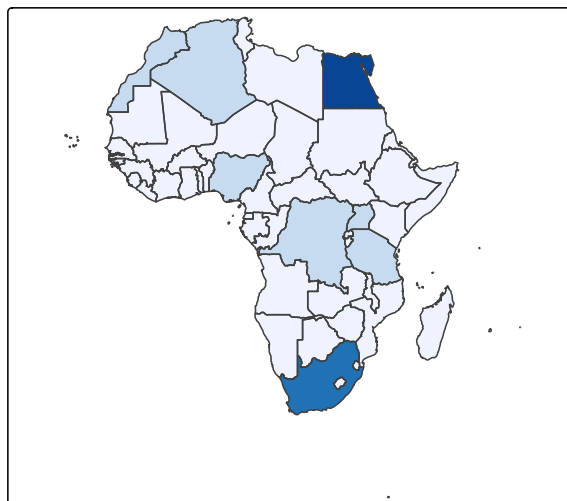
map_male <- tm_shape(africa_health) +
  tm_fill(
    "male",
    fill.scale = tm_scale(values = "brewer.blues"),
    fill.legend = tm_legend(title = "Male Cases")
  ) +
  tm_borders() +
  tm_title(text = "B. Male Counts")

# Arrange in a grid using tmap_arrange
combined <- tmap_arrange(map_inc, map_male, ncol = 2)
combined
```

A. Incidence



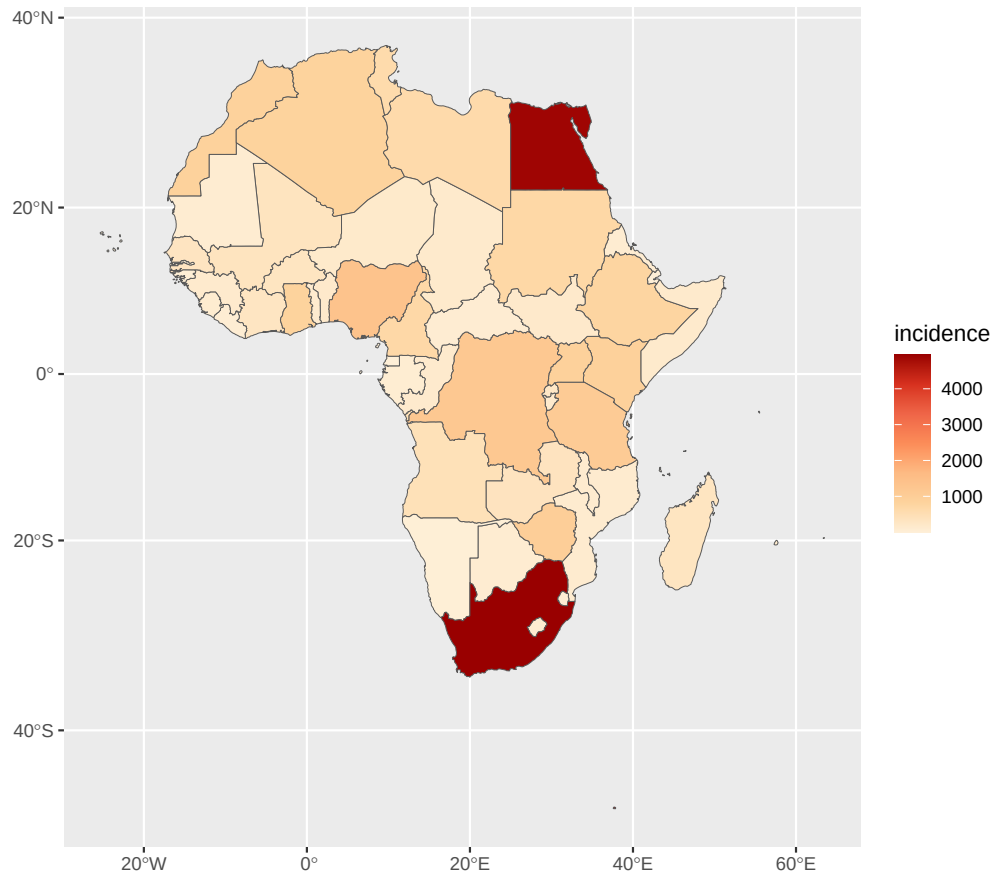
B. Male Counts



```
# mlsatial can also be used to combine the plots
# combined <- mlsatial::plot_map_grid(list(map_inc, map_male), ncol = 2)
# combined

# Export static tmap
tmap_save(
  combined,
  filename = "africa_disease_panel.png",
  width = 12, height = 6, dpi = 300
)

# Export ggplot object
gg <- ggplot(africa_health) +
  geom_sf(aes(fill = incidence)) +
  scale_fill_distiller(palette = "OrRd", direction = 1)
gg
```



```
ggsave("africa_incidence_ggplot.pdf", plot = gg, width = 10, height = 8)
```

i Combining and saving figures

- `tmap_arrange(map_inc, map_male, ncol = 2)` places two (or more) `tmap` objects side-by-side in a single figure. `mlspatial::plot_map_grid()` is a thin wrapper over it.
- `tmap_save()` writes a `tmap` figure to disk. `width/height` are in inches and `dpi` (dots per inch) controls resolution — 300 dpi is the usual print standard.
- `ggsave()` is the `ggplot2` equivalent and infers the format from the file extension (`.png`, `.pdf`, `.svg`, `.tiff`, etc.).

Key parameters: - `plot_map_grid()`: `ncol` sets the number of columns. - `tmap_save()`: DPI and dimensions control print resolution. - `ggsave()`: Supports `.png`, `.pdf`, `.svg`, `.tiff`.

Expected output: A 2-panel figure file saved to your working directory and a pdf file containing a single map.

2.7 Practical Exercise

Apply the mapping tools to a different variable and check your answers against the solution.

1. Use `plot_single_map()` to map the `male` variable with the "blues" palette and add an appropriate title.
2. Recreate that map in `ggplot2` with `geom_sf()`, using a different palette from `scale_fill_distiller()`, and add a subtitle.
3. Build a `tmap` choropleth of `female` that includes borders, a legend title, a compass, and a scale bar.
4. (*Challenge*) Simulate 50 random points with `st_sample()` (use your own seed), overlay them on an incidence map with `tm_dots()`, and give them a single colour.
5. Arrange your two `tmap` maps (incidence and male) side-by-side and save them as a PNG at 250 dpi.


```

library(sf)
library(tmap)
library(ggplot2)
library(dplyr)
library(mlspatial)

data(africa_shp, package = "mlspatial")
data(panc_incidence, package = "mlspatial")
africa_health <- mlspatial::join_data(
  africa_shp |>
    mutate(NAME = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME)),
  panc_incidence, by = "NAME"
)

# 1. Quick map of male counts
mlspatial::plot_single_map(
  sf_data = africa_health,
  var = "male",
  title = "Male Cases",
  palette = "blues"
)

# 2. ggplot version with a subtitle
ggplot(africa_health) +
  geom_sf(aes(fill = male), colour = "white", linewidth = 0.2) +
  scale_fill_distiller(palette = "Blues", direction = 1) +
  labs(title = "Male Pancreatic Cancer Cases",
       subtitle = "GBD 2021 estimates") +
  theme_minimal()

# 3. tmap choropleth of female with furniture
tm_shape(africa_health) +
  tm_fill("female",
         fill.scale = tm_scale(values = "brewer.purples"),
         fill.legend = tm_legend(title = "Female Cases")) +
  tm_borders(col = "grey40", lwd = 0.5) +
  tm_compass(position = c("right", "top")) +
  tm_scalebar(position = c("right", "bottom"))

# 4. Random points overlaid (own seed)
set.seed(5327)
pts <- st_sample(africa_health, size = 50, type = "random") |>
  st_as_sf() |>
  mutate(type = "Facilities")

tm_shape(africa_health) +
  tm_polygons("incidence",
             fill.scale = tm_scale_continuous(values = "brewer.yl_or_rd"),
             fill.legend = tm_legend(title = "Incidence"),
             col = "grey60", lwd = 0.5) +
  tm_shape(pts) +
  tm_dots(fill = "type",
         fill.scale = tm_scale_categorical(values = "darkblue"))

```

Module 2 Checklist

- ☐ Created a quick thematic map with `plot_single_map()`
- ☐ Added labels with `tm_text()`
- ☐ Built a `ggplot2` choropleth with `geom_sf()`
- ☐ Switched to interactive mode and back
- ☐ Overlaid polygon and point layers
- ☐ Exported a multi-panel figure
- ☐ Practical exercise completed

Next: Proceed to [Module 3: Spatial Autocorrelation and Cluster Analysis](#).

3 Module 3: Spatial Autocorrelation and Cluster Analysis

This module introduces the concept of spatial autocorrelation—whether disease rates in one location are similar to rates in nearby locations. You will build spatial weights matrices, compute global and local Moran’s I, detect hotspots with **Getis-Ord Gi***, and visualize the results.

What you will learn

- What spatial autocorrelation is and why it matters in epidemiology.
- How to define “neighbours” and build a spatial weights matrix.
- How to test for global clustering with Moran’s I.
- How to locate *where* clustering happens with Local Moran’s I (LISA).
- How to find hotspots/coldspots with **Getis-Ord Gi***.

Tobler’s First Law of Geography: *Everything is related to everything else, but near things are more related than distant things.* Spatial autocorrelation is the formal way to measure whether this is true for your data.

i Why spatial autocorrelation matters

If a disease were randomly distributed, every region’s rate would be independent of its neighbours. In reality, infectious diseases, environmental risk factors, and health-service access all spill across borders, so nearby regions tend to resemble each other. Testing for spatial autocorrelation tells you whether that resemblance is stronger than chance — and, if so, *where* the clusters are. This matters for two reasons: it reveals real disease hotspots, and it flags a violation of the independence assumption behind ordinary (non-spatial) regression models, which Module 4 addresses.

3.1 Build Spatial Weights Matrix

Purpose: Define which polygons are “neighbours” so we can test whether disease rates in one country resemble those in adjacent countries.

Every spatial statistic needs a definition of “**who is near whom**”. That definition is stored in a *spatial weights matrix*: for each region, a list of its neighbours and the weight given to each.

```
# Create adjacency neighbour list (queen contiguity: shares any boundary point)
nb <- poly2nb(
  africa_health,
  row.names = africa_health$NAME,
  queen = TRUE,
  snap = 1000
)

# Convert neighbour list to spatial weights list
# style = "W" = row-standardized (each row sums to 1)
# zero.policy = TRUE allows regions with no neighbors (e.g., islands)
lw <- nb2listw(nb, style = "W", zero.policy = TRUE)

# Summary of neighbour connections
summary(nb)
```

Neighbour list object:

Number of regions: 54

Number of nonzero links: 214

Percentage nonzero weights: 7.33882

Average number of links: 3.962963

6 regions with no links:

Cabo Verde, Comoros, Sao Tome and Principe, Madagascar, Mauritius,
Seychelles

7 disjoint connected subgraphs

Link number distribution:

0	1	2	3	4	5	6	7	8	9
6	2	7	9	6	7	11	3	2	1

2 least connected regions:

Gambia Lesotho with 1 link

1 most connected region:

Congo DRC with 9 links

i Reading the weights-matrix code

- `poly2nb()` builds the *neighbour list*. For polygon data the usual rule is **contiguity**: two regions are neighbours if they touch. `queen = TRUE` (queen contiguity) treats them as neighbours if they share any point or edge, like chess-queen moves; `queen = FALSE` (rook contiguity) requires a shared *edge*.
- `snap = 1000` is a distance tolerance in the units of your data. Because `africa_health` is projected in metres (EPSG:3857), this tolerates gaps up to 1,000 m between supposedly adjacent borders — a common fix for boundaries that don't quite line up.
- `row.names` attaches the country name to each entry, making the object easier to read.
- `nb2listw()` converts the neighbour list into *weights*. `style = "W"` row-standardizes, so each neighbour's weight is $1 / (\text{number of neighbours})$ and every row sums to 1. Alternatives are "B" (binary 0/1) and "C" (globally standardized).
- `zero.policy = TRUE` is essential here: island countries have no contiguous neighbours, and without this flag the functions would error on them.

Key parameters: - `poly2nb()`: `queen = TRUE` defines neighbours if they share even a single point; `queen = FALSE` (rook) requires a shared edge. - `nb2listw()`: `style = "W"` row-standardizes; "B" is binary (0/1); "C" is globally standardized.

Expected output: The `summary(nb)` reports 54 regions connected by 214 links (an average of about 4 neighbours each), with **6 regions having no neighbours** — the island nations Cabo Verde, Comoros, São Tomé and Príncipe, Madagascar, Mauritius, and Seychelles. There are 7 disconnected sub-graphs (the mainland plus six islands).

3.2 Calculate Global Moran's I

Purpose: Test whether there is **overall** spatial clustering in the disease rate across the entire study area.

Global Moran's I summarizes the whole map with a single number: are similar values generally near each other?

```
# Method 1: Direct spdep (classic approach, recommended)
global_moran <- moran.test(
  x = africa_health$incidence,
  listw = lw,
  zero.policy = TRUE,
```

```
na.action = na.exclude
)
print(global_moran)
```

Moran I test under randomisation

```
data: africa_health$incidence
weights: lw
n reduced by no-neighbour observations
```

Moran I statistic standard deviate = -0.6633, p-value = 0.7464

alternative hypothesis: greater

sample estimates:

Moran I statistic	Expectation	Variance
-0.07377293	-0.02127660	0.00626388

```
# Method 2: mlspatial wrapper (computes global + local in one call)
# autocorr_result <- compute_spatial_autocorr(
#   sf_data = africa_health,
#   values = africa_health$incidence,
#   signif = 0.05
# )
# print(autocorr_result$moran)
```

i Interpreting Moran's I

Moran's I ranges from **-1 to +1**:

- **+1** — perfect clustering: high values cluster with high, low with low.
- **0** — no spatial autocorrelation: values are arranged randomly.
- **-1** — perfect dispersion: high values sit next to low ones (a “checkerboard” pattern).

`moran.test()` compares the observed I (Index) against a null hypothesis of spatial randomness and returns a p-value. A p-value below 0.05 means the clustering is unlikely to be due to chance.

In this dataset the observed I is about -0.07 with a p-value of about 0.75.

That is close to zero and *not* significant, so there is no strong continent-wide clustering of incidence. This is an important result in itself: a global statistic can be non-significant **even when there are local clusters** (a few high-value regions cancelling out a few low-value ones). That is exactly why we turn next to *local* statistics.

Key parameters:

- `moran.test()`: Tests the null hypothesis of spatial randomness. - Statistic range: -1 (perfect dispersion) to $+1$ (perfect clustering); 0 = random.

Expected output:

- Moran's I statistic $+1$: strong positive autocorrelation (high value near high, low near low); 0 : no spatial autocorrelation (random pattern); -1 : strong negative autocorrelation (high near low, like a checkerboard pattern).
- A p-value < 0.05 indicates significant clustering, and vice versa.

3.3 Calculate Local Moran's I (LISA)

Purpose: Identify **which specific locations** contribute to the global clustering (local clusters). LISA stands for Local Indicators of Spatial Association.

While the global statistic gives one number for the whole map, LISA computes a statistic **per region**, so you can map where the clusters are.

```
# Method 1: spdep localmoran
local_mi <- localmoran(
  x = africa_health$incidence,
  listw = lw,
  zero.policy = TRUE,
  na.action = na.exclude
)
head(local_mi)
```

	Ii	E.Ii	Var.Ii	Z.Ii	Pr(z != E(Ii))
Burkina Faso	0.04214275	-0.0008535975	0.006937761	0.5162046	0.6057115
Cabo Verde	0.00000000	0.0000000000	0.000000000	NaN	NaN
Cote d'Ivoire	0.04082654	-0.0009424844	0.009386989	0.4311131	0.6663862
Gambia	0.10798905	-0.0053808375	0.289001741	0.2108859	0.8329763
Ghana	-0.10031648	-0.0023423556	0.040445809	-0.4871633	0.6261426
Guinea	0.13377548	-0.0029066090	0.023575422	0.8901883	0.3733648

```
# Attach results to sf object
africa_health$local_i <- local_mi[, "Ii"]
africa_health$local_i_p <- local_mi[, "Pr(z != E(Ii))"]
head(africa_health)
```

Simple feature collection with 6 features and 27 fields

Geometry type: GEOMETRY

Dimension: XY

Bounding box: xmin: -2823124 ymin: 484116 xmax: 266936 ymax: 1943229

Projected CRS: WGS 84 / Pseudo-Mercator

	OBJECTID	FIPS_CNTRY	ISO_2DIGIT	ISO_3DIGIT	NAME	COUNTRYAFF
1	2	UV	BF	BFA	Burkina Faso	Burkina Faso
2	3	CV	CV	CPV	Cabo Verde	Cabo Verde
3	4	IV	CI	CIV	Cote d'Ivoire	Cote d'Ivoire
4	5	GA	GM	GMB	Gambia	Gambia
5	6	GH	GH	GHA	Ghana	Ghana
6	7	GV	GN	GIN	Guinea	Guinea

	CONTINENT	TOTPOP	incidence	female	male	ageb	agec
1	Africa	20107509	330.41994	1683.1094	1868.9050	669.72101	2877.6962
2	Africa	560899	53.35342	362.2024	211.3468	93.66147	479.6228
3	Africa	24184810	320.36801	1591.7918	1852.1643	856.61193	2578.6005
4	Africa	2051363	31.35338	139.5695	197.4794	68.68145	267.6498
5	Africa	27499924	856.31466	4565.6341	4639.7485	2046.98942	7146.5049
6	Africa	12413867	163.08214	375.0091	1378.1239	336.74734	1414.2562

	agea	fageb	fagec	fagea	mageb	magec	magea
1	4.5971470	250.25241	1429.4440	3.4130101	419.46860	1448.2522	1.1841369
2	0.2650072	40.16477	321.8922	0.1455062	53.49671	157.7306	0.1195009
3	8.7436785	322.81371	1261.6862	7.2918935	533.79822	1316.9143	1.4517850
4	0.7175725	23.10264	115.9190	0.5478563	45.57881	151.7309	0.1697162
5	11.8882345	782.03099	3774.7875	8.8156430	1264.95843	3371.7174	3.0725915
6	2.1295165	59.11939	314.6618	1.2279604	277.62795	1099.5944	0.9015561

	yra	yrb	ycr	yrd	yre
1	182.37043	187.18916	193.06376	198.54795	204.27472
2	30.16095	34.07653	34.97268	35.87397	36.47930
3	158.59941	164.13910	168.28467	175.80819	182.24539
4	16.60058	17.06432	18.00591	18.29294	18.67213
5	524.74512	552.56356	578.46814	602.67088	621.47540
6	73.10342	74.94067	76.85661	78.59267	79.42589

	geometry	local_i	local_i_p
1	POLYGON ((102188.3 1231699,...	0.04214275	0.6057115
2	MULTIPOLYGON (((-2721438 16...	0.00000000	NaN
3	MULTIPOLYGON (((-350501.5 5...	0.04082654	0.6663862

```

4 POLYGON ((-1848987 1513709,... 0.10798905 0.8329763
5 POLYGON ((-352664.2 697837.... -0.10031648 0.6261426
6 POLYGON ((-884637.8 895544.... 0.13377548 0.3733648

```

```

# Method 2: mlspatial (recommended for integrated workflow)
# The wrapper already added standardized values, spatial lag, local Moran's I,
# z-scores, p-values, and cluster classification to autocorr_result$data
# lisa_sf <- autocorr_result$data
# head(lisa_sf)

```

i Reading the LISA output

`localmoran()` returns a matrix with one row per region and these columns:

- `Ii` — the local Moran statistic: its sign and size show the strength and direction of the local pattern.
- `E.Ii` — the expected value under spatial randomness (the baseline).
- `Var.Ii` — the variance (uncertainty) of that estimate.
- `Z.Ii` — the z-score (how many standard errors `Ii` is from `E.Ii`).
- `Pr(z != E(Ii))` — the p-value for that region.

A region with a small p-value (e.g. < 0.05) is a *significant* local cluster. Note that the six island countries with no neighbours receive NA here — their local statistic is undefined, which is why `zero.policy = TRUE` was needed.

The quadrant classification (High-High, Low-Low, etc.) is stored in an attribute rather than the matrix itself; we extract it in the visualization section below.

Key parameters: - `local_moran()` output is a matrix of the local statistics. `Ii` tells the strength and direction of local pattern, `E.Ii` tells the baseline under randomness, `Var.Ii` shows the estimate uncertainties (variance). `Z.Ii` is the z-score (standardized deviation) and `Pr(z != E(Ii))` is the p-value.

Expected output: - `localmoran()`: Returns `Ii` (local statistic), `E.Ii` (expected statistic), variance, z-score, and p-value for each polygon. - the `compute_spatial_autocorr()` from `mlspatial` generates the local Moran statistics in addition to the global Moran's.

3.4 Hotspot Analysis (Getis-Ord G_i^*)

Purpose: Distinguish statistically significant **hotspots** (high values surrounded by high values) from **coldspots** (low-low) using the *Getis – Ord G_i^** statistic.

Where LISA asks “is this region *correlated* with its neighbours?”, the Gi* statistic asks a subtly different question: “is this region part of a cluster of **high** (or **low**) values?” It is the standard tool for labelling hotspots and coldspots on a map.

```
# localG returns z-scores; positive = hotspot, negative = coldspot
# Note: Because we want to know if a target country combined with
# its neighbours is a hotspot or not,
# we first add country to its own neighbour list (useful for Gi*)
nb_self <- include.self(nb)
summary(nb_self)
```

Neighbour list object:

Number of regions: 54

Number of nonzero links: 268

Percentage nonzero weights: 9.190672

Average number of links: 4.962963

7 disjoint connected subgraphs

Link number distribution:

```
1  2  3  4  5  6  7  8  9 10
6  2  7  9  6  7 11  3  2  1
```

6 least connected regions:

Cabo Verde Comoros Sao Tome and Principe Madagascar Mauritius Seychelles with 1 link

1 most connected region:

Congo DRC with 10 links

```
gi_star <- localG(
  x = africa_health$incidence,
  listw = nb2listw(nb_self, style = 'W', zero.policy = TRUE),
  zero.policy = TRUE
)

# Add Gi* to dataset
africa_health <- africa_health |>
  mutate(gi_star = as.numeric(gi_star))

# Classify hotspots at alpha = 0.05 (|z| > 1.96)
# Method 1: Using the standard normal distribution cutoff
africa_health <- africa_health |>
  mutate(hotspot = case_when(
    gi_star > 1.96 ~ "Hotspot (High-High)",
    gi_star < -1.96 ~ "Coldspot (Low-Low)",
```

```

    TRUE ~ "Not significant"
  ))

# Method 2: Using the exact p-values from localG (recommended)
gi_internals <- attr(gi_star, "internals")

africa_health$gi_z <- gi_internals[, "Z(G*i)"]
africa_health$gi_p <- gi_internals[, "Pr(z != E(G*i))"]

africa_health <- africa_health |>
  mutate(
    hotspot_exact = case_when(
      gi_z > 0 & gi_p <= 0.05 ~ "Hotspot",
      gi_z < 0 & gi_p <= 0.05 ~ "Coldspot",
      TRUE ~ "Not significant"),
    hotspot_exact = factor(hotspot_exact,
                          levels = c("Coldspot", "Hotspot", "Not significant"))
  )

table(africa_health$hotspot_exact)

```

Coldspot	Hotspot	Not significant
0	3	51

i How Gi* works

- `include.self(nb)` adds each region to its *own* neighbour list. The Gi* statistic differs from LISA in that it explicitly includes the focal unit in the local sum — a region is a hotspot only if it is high **and** surrounded by highs.
- `localG()` then computes the statistic for every region. The values it returns are **z-scores**, standardised against the null hypothesis of no clustering.
- A z-score above **+1.96** marks a significant hotspot (top 2.5% of a normal distribution); below **−1.96** a significant coldspot; values in between are “not significant”. Method 1 applies this 1.96 cut-off directly, while Method 2 uses each region’s exact p-value (equivalent but slightly cleaner).
- `attr(gi_star, "internals")` is where `localG()` hides the detailed output — a matrix with columns `G*i`, `E(G*i)`, `V(G*i)`, `Z(G*i)`, and `Pr(z != E(G*i))`.
- The final `factor(..., levels = ...)` fixes the order of the categories so the legend always reads “Coldspot, Hotspot, Not significant” rather than alphabetical

order.

Key parameters: - `include.self()`: Adds exactly one neighbour to each country (itself). This helps the G_i^* calculation since it works by calculating a local sum for each country (its value plus neighbours' values). - `localG()`: Computes G_i^* which includes the focal unit in its own neighbourhood. The values calculated are Z scores. - $z > 1.96$ indicates a statistically significant hotspot; $z < -1.96$ indicates a coldspot and z near 0 is not significant.

Expected output: A frequency table showing how many countries are classified as hotspots, coldspots, or non-significant. For this dataset the result is typically **3 hotspots, 0 coldspots, and 51 non-significant** countries.

3.5 Visualize Cluster Results

Purpose: Map LISA clusters and G_i^* hotspots to communicate findings to decision-makers. The local MI values contain an attribute called `quadr` (quadrant); we can use this to visualize the local clusters.

```
# Map LISA clusters by extracting
# the quadrant and exact p-values from the local_mi
africa_health$lisa_quadrant <- attributes(local_mi)$quadr$mean

# filter the quadrants by the 0.05 significance threshold
africa_health <- africa_health |>
  mutate(
    lisa_cluster = case_when(
      local_i_p <= 0.05 ~ as.character(lisa_quadrant),
      TRUE ~ "Not significant"),
    lisa_cluster = factor(lisa_cluster,
                        levels = c("Low-Low", "High-Low", "Low-High", "High-High", "Not significant"))

# Create map
lisa_colours <- c(
  "High-High" = "red",
  "Low-Low" = "blue",
  "High-Low" = "lightpink",
  "Low-High" = "lightblue",
  "Not significant" = "grey90"
```

```

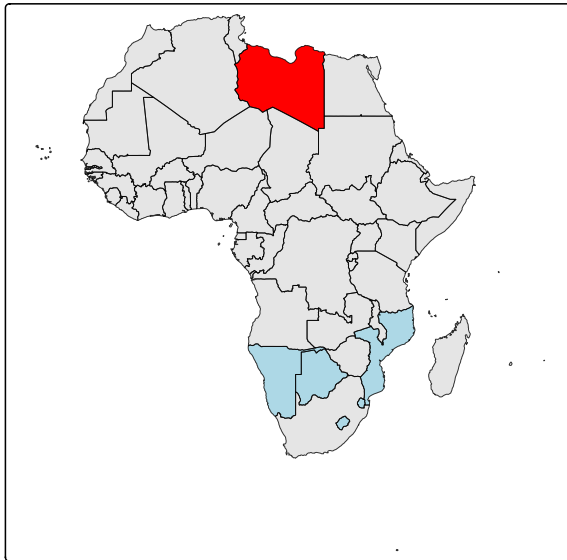
)
lisa_map <- tm_shape(africa_health) +
  tm_polygons(
    fill = "lisa_cluster",
    fill.scale = tm_scale_categorical(values = lisa_colours),
    fill.legend = tm_legend(title = "LISA Clusters (Pancreatic Incidence)",
    col = "black",
    col_alpha = 0.8,
    lwd = 0.5
  ) +
  tm_title("Local Moran's I (LISA) Cluster Map")

# Map Gi* hotspots
Gi_colours <- c(
  "Hotspot" = "red",
  "Coldspot" = "blue",
  "Not significant" = 'grey90'
)
hotspot_map <- tm_shape(africa_health) +
  tm_polygons(
    fill = "hotspot_exact",
    fill.scale = tm_scale(values = Gi_colours),
    fill.legend = tm_legend(title = "Gi* Classification")
  ) +
  tm_title("Getis-Ord Gi* Hotspots")

hotspots <- tmap_arrange(lisa_map, hotspot_map, ncol = 2)
hotspots

```

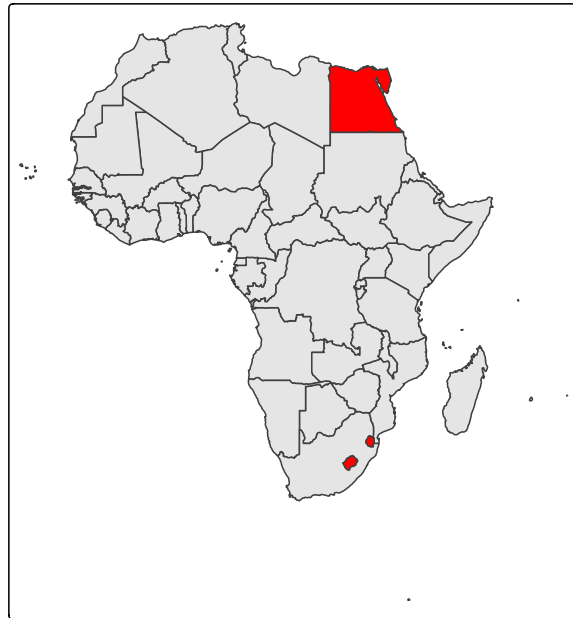
Local Moran's I (LISA) Cluster Map



LISA Clusters (Pancreatic Incidence)



Getis-Ord Gi* Hotspots



Gi* Classification



```
tmap_save(
  hotspots,
  filename = "africa_hotspot_panel.png",
  width = 12, height = 6, dpi = 300
)
```

i Reading the visualization code

- `attributes(local_mi)$quadr$mean` extracts the quadrant classification from the `localmoran()` output. The four quadrants are named by the region's value and its neighbours' values: **High-High** (a hot cluster), **Low-Low** (a cold cluster), **High-Low** (a high region surrounded by lows), and **Low-High** (the inverse).
- `case_when(local_i_p <= 0.05 ~ ..., TRUE ~ "Not significant")` keeps only the quadrants that are statistically significant, and relabels the rest. (Regions whose p-value is NA — the islands — fall through to "Not significant".)
- The named colour vector `lisa_colours` maps each category to a fixed colour, and

- `tm_scale_categorical()` applies it. Giving the `factor()` an explicit `levels` order guarantees the legend order matches the colours.
- `tmap_arrange(..., ncol = 2)` places the two maps side by side, and `tmap_save()` writes the panel to a PNG at 300 dpi.

Expected output: A two-panel map. The left panel shows LISA clusters; the right panel shows G_i^* hotspots and coldspots.

3.6 Practical Exercise

Re-run the whole workflow for a different variable — the male case counts (`male`) — and check your answers against the solution.

1. Build a **rook** contiguity neighbour list (`queen = FALSE`, keeping `snap = 1000`). How does the total number of links compare with the queen version above?
2. Compute global Moran's I for `male` and state whether clustering is significant at the 0.05 level.
3. Compute local Moran's I for `male`. How many regions have a significant ($p < 0.05$) local statistic?
4. Run Getis-Ord G_i^* on `male` and report how many hotspots and coldspots you find.
5. (*Challenge*) Map the G_i^* result for `male` using `tm_polygons()` with your own colour scheme.


```

library(sf)
library(spdep)
library(dplyr)
library(mlspatial)
library(tmap)

data(africa_shp, package = "mlspatial")
data(panc_incidence, package = "mlspatial")
africa_health <- join_data(
  africa_shp |> mutate(NAME = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME)),
  panc_incidence, by = "NAME"
)
africa_health <- st_make_valid(africa_health)

# 1. Rook contiguity
nb_rook <- poly2nb(africa_health, queen = FALSE, snap = 1000)
summary(nb_rook) # fewer links than the queen version (shared edges only)

# 2. Global Moran's I for male
lw_rook <- nb2listw(nb_rook, style = "W", zero.policy = TRUE)
moran.test(africa_health$male, listw = lw_rook,
  zero.policy = TRUE, na.action = na.exclude)

# 3. Local Moran's I for male
lm_male <- localmoran(africa_health$male, listw = lw_rook,
  zero.policy = TRUE, na.action = na.exclude)
sum(lm_male[, "Pr(z != E(Ii))"] <= 0.05, na.rm = TRUE)

# 4. Getis-Ord Gi* for male
nb_self <- include.self(nb_rook)
gi_male <- localG(africa_health$male,
  listw = nb2listw(nb_self, style = "W", zero.policy = TRUE),
  zero.policy = TRUE)
gi_int <- attr(gi_male, "internals")
africa_health$gi_z <- gi_int[, "Z(G*i)"]
africa_health$gi_p <- gi_int[, "Pr(z != E(G*i))"]
africa_health <- africa_health |>
  mutate(hotspot = case_when(
    gi_z > 0 & gi_p <= 0.05 ~ "Hotspot",
    gi_z < 0 & gi_p <= 0.05 ~ "Coldspot",
    TRUE ~ "Not significant"))
table(africa_health$hotspot)

# 5. Map the Gi* result
cols <- c("Hotspot" = "red", "Coldspot" = "blue", "Not significant" = "grey90")
tm_shape(africa_health) +
  tm_polygons(fill = "hotspot",
    fill.scale = tm_scale_categorical(values = cols),
    fill.legend = tm_legend(title = "Gi* (male)")) +
  tm_title("Getis-Ord Gi* Hotspots - Male Cases")

```

Module 3 Checklist

- ☐ Built a spatial weights matrix with `poly2nb()` and `nb2listw()`
- ☐ Computed Global Moran's I and interpreted the p-value
- ☐ Computed Local Moran's I (LISA) and identified cluster types
- ☐ Calculated Getis-Ord G_i^* and classified hotspots/coldspots
- ☐ Visualized both LISA and G_i^* results side-by-side
- ☐ Practical exercise completed

Next: Proceed to [Module 4: Spatial Regression and Environmental Factor Analysis](#).

4 Module 4: Spatial Regression and Epidemiological Risk Factor Analysis

This module connects health outcomes to lifestyle/genetic drivers. You will simulate covariates, build a global OLS model, test its residuals for spatial autocorrelation, and then fit a Geographically Weighted Regression (GWR) to capture spatially varying relationships.

What you will learn

- How to assemble risk-factor covariates for a regression.
- How to fit and interpret an ordinary least squares (OLS) model.
- How to spot **data leakage** — a perfect fit that means the outcome was derived from its own predictors.
- Why we model *rates* rather than raw counts.
- How to test OLS residuals for spatial autocorrelation.
- What Geographically Weighted Regression (GWR) is, and how to judge whether it is actually an improvement.

i Where this module sits

Modules 2 and 3 mapped the disease and measured its spatial clustering. Here we ask a causal question: *which factors drive the disease rate?* Ordinary regression assumes each observation is independent, but Module 3 showed spatial data can violate that. The workflow below is the standard spatial-epidemiology sequence: fit a global model, check its residuals for spatial structure, and only then reach for a local (GWR) model if the data justify it.

4.1 Simulate Risk-Factor Covariates

Purpose: Link health outcomes to known epidemiological risk factors for pancreatic cancer (smoking, obesity, diabetes, alcohol use, family history, urbanisation) for use in the spatial regression exercise.

For this tutorial, we **simulate** six covariates directly at the country level so the code is fully runnable without downloading external datasets. Because `africa_health` is a polygon layer where each row already represents one country (like `incidence`, `TOTPOP`, etc.), we do not need raster data — we can generate one random value per row and append it directly with `mutate()`.

In real research, replace these random draws with actual country-level estimates from sources such as the WHO Global Health Observatory (smoking, obesity, diabetes prevalence), IARC GLOBOCAN risk-factor tables, or national health surveys. These are typically small CSV/Excel downloads, joined to `africa_health` by country ISO code (`ISO_3DIGIT`) rather than simulated.

```
set.seed(898) # reproducibility

n <- nrow(africa_health)

africa_health <- africa_health |>
  mutate(
    # Smoking prevalence (% of adult population) - WHO African region estimates
    # typically range ~10-25%; strongest modifiable risk factor for pancreatic cancer
    smoking_prev = round(runif(n, min = 8, max = 28), 1),

    # Obesity prevalence (% adults, BMI >= 30) - WHO Africa region estimates
    # range roughly 5-30% depending on urbanization/country
    obesity_prev = round(runif(n, min = 5, max = 30), 1),

    # Type 2 diabetes prevalence (%) - longstanding diabetes is a well-established
    # pancreatic cancer risk factor; IDF Africa estimates ~3-10% typically
    diabetes_prev = round(runif(n, min = 2, max = 11), 1),

    # Alcohol consumption (litres pure alcohol per capita per year) -
    # heavy long-term use linked to chronic pancreatitis -> elevated risk
    alcohol_per_capita = round(runif(n, min = 0.5, max = 12), 1),

    # Family history / genetic predisposition proxy (% of cases with
    # first-degree relative history) - literature suggests ~5-10% of
    # pancreatic cancers have a hereditary component
    family_history_prev = round(runif(n, min = 3, max = 10), 1),

    # Urbanization (%) - proxy for dietary/lifestyle transition,
    # often used as a covariate for NCD risk in African epidemiology
    urbanization_pct = round(runif(n, min = 15, max = 85), 1)
  )
```

```
head(africa_health[c("NAME", "smoking_prev", "obesity_prev",
                     "diabetes_prev", "alcohol_per_capita",
                     "family_history_prev", "urbanization_pct")])
```

Simple feature collection with 6 features and 7 fields

Geometry type: GEOMETRY

Dimension: XY

Bounding box: xmin: -2823124 ymin: 484116 xmax: 266936 ymax: 1943229

Projected CRS: WGS 84 / Pseudo-Mercator

	NAME	smoking_prev	obesity_prev	diabetes_prev	alcohol_per_capita
1	Burkina Faso	13.6	13.7	7.0	2.9
2	Cabo Verde	24.1	6.6	4.3	1.5
3	Cote d'Ivoire	17.3	26.3	4.0	8.6
4	Gambia	20.1	9.9	8.2	8.6
5	Ghana	17.1	11.5	3.1	6.4
6	Guinea	27.4	10.8	5.7	10.4

	family_history_prev	urbanization_pct	geometry
1	9.7	73.2	POLYGON (((102188.3 1231699,...
2	6.7	54.2	MULTIPOLYGON (((-2721438 16...
3	9.2	45.8	MULTIPOLYGON (((-350501.5 5...
4	3.9	68.4	POLYGON ((-1848987 1513709,...
5	7.8	49.0	POLYGON ((-352664.2 697837....
6	3.4	69.4	POLYGON ((-884637.8 895544....

i Why we simulate (and why that matters for interpretation)

- `set.seed(898)` fixes the random-number generator so the “random” covariates are the same every time the document is rendered — essential for a reproducible tutorial.
- `runif(n, min, max)` draws `n` values from a uniform distribution between `min` and `max`, one per country. `round(..., 1)` keeps them to one decimal place.
- These covariates are simulated **independently of incidence and of each other** — they are *not* designed to correlate with pancreatic cancer rates. This is intentional: the goal is to demonstrate the regression *pipeline* (data structure, model syntax, diagnostics), not to produce a real epidemiological finding. As you will see, this choice turns out to be a useful teaching device.

Key parameters: - `runif(n, min, max)`: draws `n` random values from a uniform distribution bounded to a realistic range for each covariate, loosely based on published prevalence ranges across African countries.

Expected output: Six new columns appended to `africa_health`:

Column	Description	Simulated range
<code>smoking_prev</code>	Adult smoking prevalence (%)	8–28%
<code>obesity_prev</code>	Adult obesity prevalence, BMI ≥ 30 (%)	5–30%
<code>diabetes_prev</code>	Type 2 diabetes prevalence (%)	2–11%
<code>alcohol_per_capita</code>	Alcohol consumption (litres pure alcohol/year)	0.5–12
<code>family_history_prev</code>	Cases with first-degree relative history (%)	3–10%
<code>urbanization_pct</code>	Urban population (%)	15–85%

4.2 Build Ordinary Least Squares (OLS) Regression

Purpose: Establish a global (non-spatial) baseline model relating pancreatic cancer incidence to lifestyle, genetic and demographic risk factors.

```
ols_model <- lm(  
  incidence ~ smoking_prev + obesity_prev + diabetes_prev +  
    alcohol_per_capita + family_history_prev + urbanization_pct +  
    female + male + ageb + agec,  
  data = africa_health  
)  
summary(ols_model)
```

Call:

```
lm(formula = incidence ~ smoking_prev + obesity_prev + diabetes_prev +  
    alcohol_per_capita + family_history_prev + urbanization_pct +  
    female + male + ageb + agec, data = africa_health)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.601e-12	-1.500e-13	2.316e-14	2.169e-13	7.935e-13

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-2.950e-13	4.884e-13	-6.040e-01	0.549105
smoking_prev	-1.345e-14	1.124e-14	-1.196e+00	0.238171
obesity_prev	-6.195e-16	9.145e-15	-6.800e-02	0.946304
diabetes_prev	6.421e-14	3.072e-14	2.090e+00	0.042536 *
alcohol_per_capita	-9.800e-15	2.052e-14	-4.780e-01	0.635292
family_history_prev	-1.106e-14	2.899e-14	-3.820e-01	0.704643
urbanization_pct	-4.111e-16	3.459e-15	-1.190e-01	0.905951
female	9.302e-02	9.252e-15	1.005e+13	< 2e-16 ***
male	9.302e-02	9.279e-15	1.003e+13	< 2e-16 ***
ageb	3.896e-14	9.366e-15	4.160e+00	0.000149 ***
agec	3.865e-14	9.251e-15	4.178e+00	0.000141 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.452e-13 on 43 degrees of freedom

Multiple R-squared: 1, Adjusted R-squared: 1

F-statistic: 2.36e+31 on 10 and 43 DF, p-value: < 2.2e-16

Reading an lm() summary

The formula `response ~ predictor1 + predictor2 + ...` tells `lm()` what to fit. The `summary()` then reports, for each predictor:

- **Estimate** — the expected change in the outcome for a one-unit increase in that predictor, holding the others fixed.
- **Std. Error / t value / Pr(>|t|)** — how precisely the coefficient is estimated and whether it differs from zero.
- **R-squared** — the proportion of variance in the outcome explained by the model (0 to 1).
- **F-statistic** — an overall test that at least one predictor matters.

A positive coefficient means higher predictor → higher expected incidence; negative means the reverse.

Key parameters: - Formula syntax: `response ~ predictor1 + predictor2 + ...` - Interpretation: Positive coefficients indicate higher expected incidence with higher predictor values.

Expected output: A standard `lm` summary showing coefficients, standard errors, t-values, and p-values. R^2 indicates the proportion of variance explained globally.

 A red flag: a *perfect* fit

This first model prints a warning message and returns an R^2 of exactly **1** with an astronomical F-statistic. That is not a spatial-regression breakthrough — it is a classic case of **data leakage**. A model that fits perfectly is almost always telling you that the outcome is mathematically derived from one or more of its own predictors. The next section diagnoses it.

4.3 Diagnosing Data Leakage

The first **OLS** model returned **R-squared: 1**, an F-statistic of **2.36e+31** and an explicit warning — “*essentially perfect fit: summary may be unreliable*”. A model that fits perfectly is almost always telling you that the outcome is mathematically derived from one or more of its own predictors. This is called **data leakage**.

Step 1: Inspect the coefficients. Looking at the model summary, two variables stood out immediately:

```
female    9.302e-02    t = 1.005e+13    p < 2e-16
male      9.302e-02    t = 1.003e+13    p < 2e-16
```

Both coefficients are identical to four decimal figures, with t-values around 10^{13} . T-values that enormous are a signature of the model detecting an algebraic identity, not a strong effect.

Step 2: Test the suspected identity directly.

```
cor(africa_health$incidence, africa_health$female + africa_health$male)
```

```
[1] 1
```

```
head(africa_health$incidence / (africa_health$female + africa_health$male))
```

```
[1] 0.09302326 0.09302326 0.09302326 0.09302326 0.09302326 0.09302326
```

A correlation of 1 between `incidence` and `female + male` confirms that `incidence` is a fixed linear function of `female + male`. Dividing one by the other reveals the exact relationship: every single country returns the exact constant 0.09302326, meaning `incidence` was computed directly from `male` and `female` (or they share a common upstream calculation).

Including `male` and `female` as predictors makes it **mathematically circular** — the model is not learning but rather reversing an arithmetic formula.

Step 3: Screen every numeric variable for leakage.

```
# Check all numeric columns systematically against incidence
numeric_cols <- africa_health |>
  st_drop_geometry() |>
  select(where(is.numeric)) |>
  select(-incidence) |>                                # exclude the outcome itself
  names()

cor_check <- sapply(numeric_cols, function(col) {
  cor(africa_health$incidence, africa_health[[col]], use = "complete.obs")
})

sort(abs(cor_check), decreasing = TRUE)
```

agec	ageb	yra	yrb
0.999776035	0.997363274	0.994162603	0.993023880
yrd	mageb	ycr	male
0.991366013	0.991300624	0.991276110	0.990157443
yre	magec	fageb	female
0.989764734	0.987751703	0.985364466	0.982334295
fagec	magea	agea	fagea
0.979661775	0.863353711	0.751522574	0.594702968
TOTPOP	diabetes_prev	OBJECTID	obesity_prev
0.568333472	0.195570160	0.190926164	0.134111883
family_history_prev	alcohol_per_capita	urbanization_pct	smoking_prev
0.120046661	0.082419609	0.017524733	0.005253558

i What “data leakage” means and how we caught it

- **Data leakage** occurs when information from the outcome (or its future) is used to *build* the predictors. The model then appears brilliant but has learned nothing — it is just recovering an identity.
- Step 1 spotted the tell-tale signs: identical coefficients and implausibly large t-values.
- Step 2 confirmed it: `cor()` returned exactly 1, and the ratio `incidence / (female`

- + male) was a constant for every country.
- Step 3 generalised the search. `st_drop_geometry()` removes the spatial column (which is not numeric); `select(where(is.numeric))` keeps only numeric columns; `select(-incidence)` drops the outcome; then `sapply()` computes the correlation of each remaining column with `incidence`. Sorting by absolute value surfaces every leaked variable at once.

The output shows the leakage extends well beyond `female/male`. Almost every variable with `|r| > 0.85` shares a consistent naming pattern: age-banded (`age*`), year-indexed (`yr*`) and male/female-stratified (`m*/f*`). This strongly indicates they are all components of the same underlying case-count calculation that produced `incidence`.

4.4 Refined Model

Based on the correlation screen, we exclude every variable structurally tied to the incidence calculation along with the non-informative row identifier. In their place we retain our six simulated risk-factor covariates.

Another thing worth mentioning in epidemiology is the raw incidence counts. We rarely run models on raw count data. Instead we will calculate an **incidence rate** (e.g. cases per 100,000 people) before modelling.

```
africa_health <- africa_health |>
  mutate(incidence_rate = (incidence / TOTPOP) * 100000)

ols_refined <- lm(
  incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
    alcohol_per_capita + family_history_prev + urbanization_pct,
  data = africa_health
)
summary(ols_refined)
```

Call:

```
lm(formula = incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
    alcohol_per_capita + family_history_prev + urbanization_pct,
    data = africa_health)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.9009	-1.5714	-0.7732	0.8217	8.7829

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	9.33286	3.00583	3.105	0.00322 **
smoking_prev	0.02596	0.07047	0.368	0.71426
obesity_prev	-0.10385	0.05500	-1.888	0.06520 .
diabetes_prev	-0.17452	0.18733	-0.932	0.35628
alcohol_per_capita	-0.17127	0.12655	-1.353	0.18241
family_history_prev	-0.08952	0.17952	-0.499	0.62036
urbanization_pct	-0.04129	0.02130	-1.938	0.05859 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.807 on 47 degrees of freedom

Multiple R-squared: 0.1502, Adjusted R-squared: 0.04166

F-statistic: 1.384 on 6 and 47 DF, p-value: 0.2408

i Rates vs counts

Raw case counts are dominated by population size: a big country has more cases almost by definition. Dividing by population — $(\text{incidence} / \text{TOTPOP}) * 100000$ — standardises the outcome to “cases per 100,000 people”, making countries comparable. This is the same reasoning behind the age-standardised rates used in published cancer statistics.

With the leaked columns removed, this refined model has an R^2 near 0.15 and no significant predictors. Because the covariates were pure random noise, OLS did exactly what it should: it found **no** real relationship. The two variables that show marginal significance (`obesity_prev`, `urbanization_pct`) are chance findings — their signs are even biologically backwards (negative), which is what random noise looks like.

This output provides an overall statistically insignificant model. Because the simulated covariates were generated purely as random noise (`runif`), the OLS model did exactly what it is supposed to do. It shows no statistical relationship between these covariates and the target variable.

Although all the covariates are randomly generated, `obesity_prev` and `urbanization_pct` are showing marginal significance. This is purely by chance, because we can confirm from their estimate direction (-0.103 and -0.041) that the math contradicts the biology; higher obesity and urbanisation should increase pancreatic cancer risk.

4.5 Test for Spatial Residual Autocorrelation

Purpose: Check whether the OLS model's residuals are spatially random. The core assumption of OLS is that its errors (residuals) are randomly distributed. If significant autocorrelation remains, OLS standard errors are biased and a spatial model (or GWR) is needed.

In spatial epidemiology, if our model is missing a key geographical variable, the model's errors will cluster together geographically. This is called **spatial autocorrelation of residuals**.

```
residual_moran <- lm.morantest(  
  model = ols_refined,  
  listw = lw,  
  zero.policy = TRUE  
)  
print(residual_moran)
```

Global Moran I for regression residuals

data:

```
model: lm(formula = incidence_rate ~ smoking_prev + obesity_prev +  
diabetes_prev + alcohol_per_capita + family_history_prev +  
urbanization_pct, data = africa_health)
```

weights: lw

Moran I statistic standard deviate = 1.5133, p-value = 0.0651

alternative hypothesis: greater

sample estimates:

Observed Moran I	Expectation	Variance
0.125604953	-0.023885847	0.009758588

i The zero-neighbour warning, and what `zero.policy = TRUE` does

Six of the 54 units are islands that share no border with any other unit in the layer: Cabo Verde, Comoros, Sao Tome and Principe, Madagascar, Mauritius and Seychelles. `poly2nb()` warns about this and reports 7 sub-graphs — the connected mainland block of 48, plus the six islands. The warning is expected: these are genuine islands

A unit with no neighbours has an undefined spatial lag, so `spdep` has to be told what to do with those rows. `zero.policy = TRUE` keeps them in the analysis and sets their lag to 0. Setting it to `FALSE` fails instead, and not always in the same way: `nb2listw()` stops with an `Empty neighbour sets found` error, `moran.test()` stops with a `regions with no neighbours found` error, and `lm.morantest()` quietly returns NA. That is why

the weights object and every test in this module pass `zero.policy = TRUE`. The islands still enter the statistic through their own residuals, but they appear in no other unit's neighbour list. They add nothing to the numerator of Moran's I, yet still count in the denominator (the sum of squared deviations), so keeping them in pulls I slightly towards zero.

i Why test the residuals, not the outcome

The **residual** for each country is the part of its rate the model *could not* explain: **residual = observed - fitted**. If those leftovers are spatially clustered (nearby countries share similar errors), the model is missing a spatially structured cause, and its p-values are unreliable.

`lm.morantest()` applies Moran's I (from Module 3) to the residuals instead of the raw outcome. The null hypothesis is that the residuals are randomly distributed across space.

Key parameters: - `lm.morantest()`: Applies Moran's I to the regression residuals rather than the raw outcome.

Expected output: - If $p < 0.05$, residuals are spatially autocorrelated, so OLS is likely misspecified and a spatial model (like GWR) is worth trying — but whether it actually helps is settled by comparing AICc (see below), not by the test alone. - If $p > 0.05$, the residuals are not *significantly* autocorrelated — but a p-value close to 0.05 (say 0.06) signals weak, inconclusive spatial structure, not genuinely random residuals.

Here the observed Moran's I of the residuals is **+0.126** against an expectation of **-0.024**, with a p-value of **0.065**. That p-value is one-sided (**alternative hypothesis: greater**): it tests whether the residuals are *more* clustered than chance, not whether they differ from chance in either direction.

At the 5% level we do **not** reject the null hypothesis, so the residuals are not *significantly* spatially autocorrelated. But 0.065 is a borderline result rather than a comfortable one: the observed I is positive — nearby countries do share similar errors — and it misses the conventional threshold only narrowly. The defensible reading is “weak, inconclusive spatial structure in the residuals”, not “residuals are spatially random”.

4.6 Geographically Weighted Regression (GWR)

Purpose: Fit local regression models that allow the relationship between cancer and lifestyle/genetic risk factors to vary across space (spatial non-stationarity).

Ordinary regression fits **one** equation for the whole continent. GWR fits a separate weighted regression at every location, giving nearby points more influence, so each country gets its own set of coefficients.

```
# Step 1: Find optimal adaptive bandwidth (number of nearest neighbours)
# This uses cross-validation to find best fit
bw_adapt <- bw.gwr(
  formula = incidence_rate ~ smoking_prev + obesity_prev +
    diabetes_prev + alcohol_per_capita +
    family_history_prev + urbanization_pct,
  data = africa_health,
  approach = "CV",      # Cross-validation approach
  kernel = "bisquare",  # the shape of the spatial weight decay
  adaptive = TRUE       # TRUE = find optimal no. of neighbours. FALSE = find distance
)
```

```
Adaptive bandwidth: 41 CV score: 476.0228
Adaptive bandwidth: 33 CV score: 463.883
Adaptive bandwidth: 28 CV score: 472.4997
Adaptive bandwidth: 36 CV score: 464.0033
Adaptive bandwidth: 31 CV score: 466.1091
Adaptive bandwidth: 34 CV score: 463.8069
Adaptive bandwidth: 35 CV score: 463.2405
Adaptive bandwidth: 35 CV score: 463.2405
```

```
cat("Optimal adaptive bandwidth:", bw_adapt, "neighbors\n")
```

Optimal adaptive bandwidth: 35 neighbors

```
# Step 2: Fit GWR model
gwr_model <- gwr.basic(
  formula = incidence_rate ~ smoking_prev + obesity_prev +
    diabetes_prev + alcohol_per_capita +
    family_history_prev + urbanization_pct,
  data = africa_health,
  bw = bw_adapt,
  kernel = "bisquare",
  adaptive = TRUE
)

# Print summary (includes global OLS + local GWR diagnostics)
print(gwr_model)
```

```

*****
*                               Package   GWmodel                               *
*****
Program starts at: 2026-09-27 13:41:02.14502
Call:
gwr.basic(formula = incidence_rate ~ smoking_prev + obesity_prev +
  diabetes_prev + alcohol_per_capita + family_history_prev +
  urbanization_pct, data = africa_health, bw = bw_adapt, kernel = "bisquare",
  adaptive = TRUE)

Dependent (y) variable:  incidence_rate
Independent variables:  smoking_prev obesity_prev diabetes_prev alcohol_per_capita family
Number of data points: 54
*****
*                               Results of Global Regression                               *
*****

Call:
lm(formula = formula, data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-3.9009 -1.5714 -0.7732  0.8217  8.7829

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)      9.33286    3.00583   3.105  0.00322 **
smoking_prev      0.02596    0.07047   0.368  0.71426
obesity_prev     -0.10385    0.05500  -1.888  0.06520 .
diabetes_prev    -0.17452    0.18733  -0.932  0.35628
alcohol_per_capita -0.17127    0.12655  -1.353  0.18241
family_history_prev -0.08952    0.17952  -0.499  0.62036
urbanization_pct  -0.04129    0.02130  -1.938  0.05859 .

---Significance stars
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 2.807 on 47 degrees of freedom
Multiple R-squared:  0.1502
Adjusted R-squared:  0.04166
F-statistic: 1.384 on 6 and 47 DF,  p-value: 0.2408
***Extra Diagnostic information
Residual sum of squares: 370.3139
Sigma(hat): 2.668599

```

```

AIC: 273.2152
AICc: 276.4152
BIC: 267.0389
*****
*           Results of Geographically Weighted Regression           *
*****

*****Model calibration information*****
Kernel function: bisquare
Adaptive bandwidth: 35 (number of nearest neighbours)
Regression points: the same locations as observations are used.
Distance metric: Euclidean distance metric is used.

*****Summary of GWR coefficient estimates:*****
               Min.      1st Qu.      Median      3rd Qu.      Max.
Intercept      2.5414463  5.7074985  7.2222933 10.3176946 15.2724
smoking_prev   -0.1729309 -0.0182621  0.0121007  0.0870956  0.1121
obesity_prev   -0.2135095 -0.1292391 -0.0950674 -0.0555409  0.0517
diabetes_prev  -0.6432196 -0.2850108 -0.1078285  0.0331103  0.2252
alcohol_per_capita -0.5719018 -0.1832194 -0.1390848 -0.0851088  0.1293
family_history_prev -0.2341325 -0.1174264 -0.0622756 -0.0099285  0.0802
urbanization_pct -0.0772384 -0.0509485 -0.0236476 -0.0171660 -0.0028
*****Diagnostic information*****
Number of data points: 54
Effective number of parameters (2trace(S) - trace(S'S)): 25.72294
Effective degrees of freedom (n-2trace(S) + trace(S'S)): 28.27706
AICc (GWR book, Fotheringham, et al. 2002, p. 61, eq 2.33): 294.6998
AIC (GWR book, Fotheringham, et al. 2002,GWR p. 96, eq. 4.22): 239.3899
BIC (GWR book, Fotheringham, et al. 2002,GWR p. 61, eq. 2.34): 247.9558
Residual sum of squares: 180.6605
R-square value: 0.5853944
Adjusted R-square value: 0.1944111

*****
Program stops at: 2026-09-27 13:41:02.161045

```

i How GWR works, step by step

- **Bandwidth** is the size of the local “window”. `bw.gwr()` searches over bandwidths and picks the one that minimises cross-validation error (`approach = "CV"`). With `adaptive = TRUE`, the bandwidth is a *number of nearest neighbours* (35 here) rather than a distance — the right choice for irregularly sized countries.

- **Kernel** is the weighting function. "bisquare" gives full weight to near neighbours and drops smoothly to zero at the edge of the window, creating clean local windows. ("gaussian" is the other common choice; it never quite reaches zero.)
- `gwr.basic()` then fits a weighted OLS at every polygon centroid. The result is one coefficient per predictor **per country** — hence the coefficients are reported as ranges.

Key parameters: - `bw.gwr(): adaptive = TRUE` finds a bandwidth in number of neighbours rather than distance units. Better for irregularly sized polygons. - `kernel = "bisquare"`: Weights drop to zero beyond the bandwidth, creating clean local windows. - `gwr.basic()`: Fits a separate weighted OLS at every polygon centroid.

Expected output: - Bandwidth value (35 neighbours for country-level Africa data). - GWR diagnostics showing local R^2 , coefficient ranges, and AICc values vs global OLS.

i Judging whether GWR is worth it

A bandwidth of 35 is huge for a study area of only 54 countries — the model must look at over half the continent to make a single local prediction. The coefficient range for `diabetes_prev` spans roughly -0.64 to $+0.23$, suggesting the relationship *appears* to flip sign in places. But this is simulated noise, so the model is chasing nothing.

The honest way to judge GWR is **AICc** (lower is better), not R^2 — GWR almost always inflates R^2 because it fits many more parameters. Here the AICc *rose* from 276.4 for OLS to 294.7 for GWR. The local model does not earn back the parameters it spends, so on this evidence it is the worse model — a useful negative result.

The residual Moran test is only weak corroboration here ($I = 0.126$, $p = 0.065$), and its verdict is specific to this covariate set — see below.

A bandwidth of 35 is huge for a spatial region of 54 countries. The model is forced to look at over half a continent to make a single prediction. The GWR coefficient estimates are given as a range because they change depending on where you are on the map. In the global OLS the relationship for `diabetes_prev` is negative, but GWR reveals that it changes in some part of the continent (min -0.64 and max 0.23). This shows the relationship is unstable across space (but remember this is simulated data, so the model is just chasing random noise).

The GWR R^2 looks very high but that should not be the metric of interest, because it is almost always high for GWR. The parameter of interest is the AICc (lower is better). Even though the GWR artificially inflated the R^2 , the AICc went up from 276.4 in the OLS to 294.7 — the GWR is the worse model once you account for the number of parameters it spends.

Keep the scope of that verdict in mind. It is a statement about *this* six-covariate specification, not a general finding that spatial regression is unnecessary. Two things narrow it. First, the covariates were simulated at random, so there is no spatially varying relationship for GWR to recover — the local model can only chase noise. Second, the residual evidence is borderline

rather than clean: the global model leaves $I = 0.126$ ($p = 0.065$), which fails to reach significance but is not the comfortable “no clustering at all” result that would make GWR moot. Change the covariate set and the balance can shift the other way — the two-covariate model in the exercise below leaves much stronger residual clustering ($I = 0.217$, $p = 0.007$), which is precisely the situation GWR is designed for. The practical lesson is to let the AICc decide for each specification instead of assuming that a residual test has ruled spatial models out in advance.

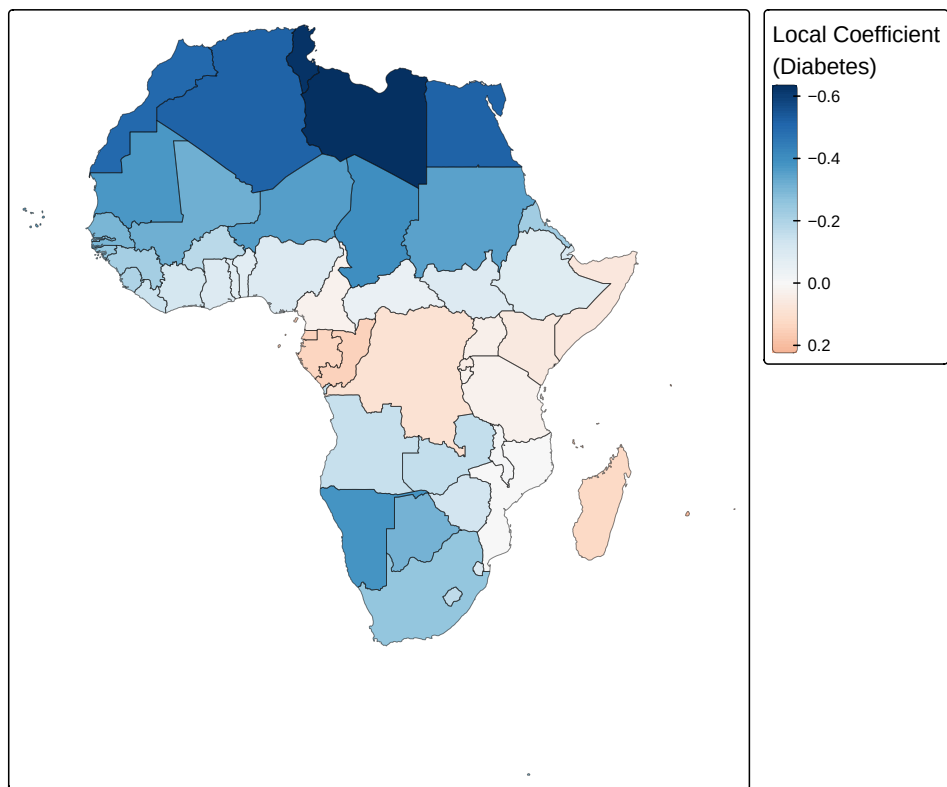
4.7 Interpret and Visualize GWR Results

Purpose: Map local coefficient surfaces to see **where** each predictor matters most.

```
# gwr_model$SDF is an sf object containing local coefficients
gwr_sf <- gwr_model$SDF

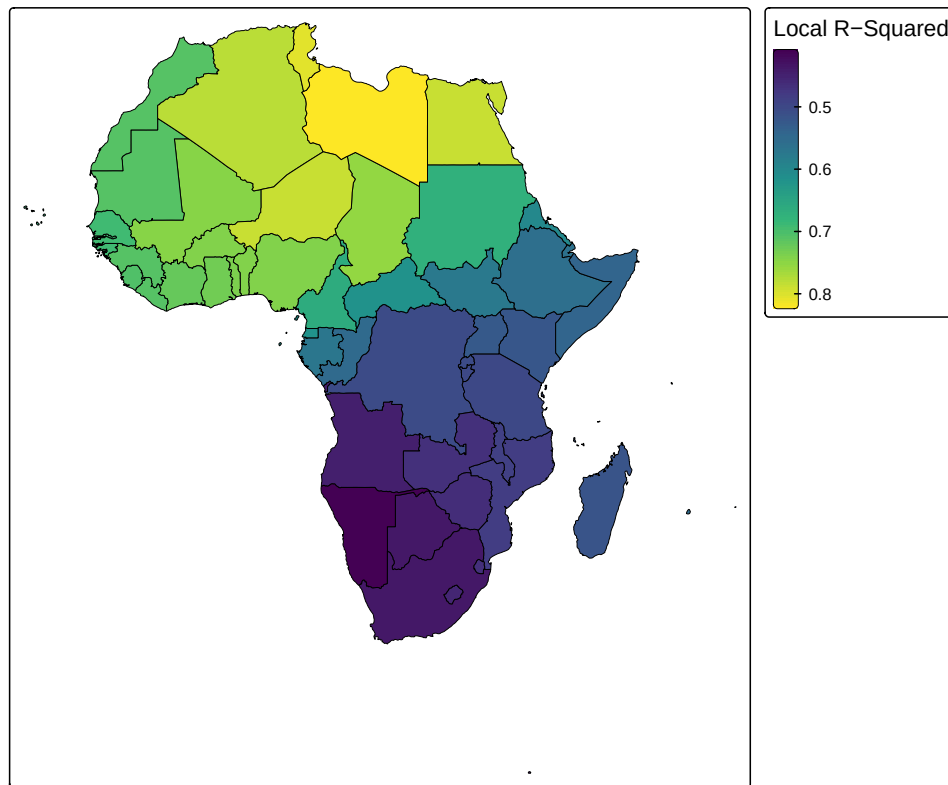
# Map the spatially varying coefficient for diabetes
tm_shape(gwr_sf) +
  tm_polygons(
    fill = "diabetes_prev",
    fill.scale = tm_scale_continuous(
      values = "-brewer.rd_bu",      # Negative RdBu puts red at positive and blue at negative
      midpoint = 0                    # Forces the white/neutral colour to sit exactly at zero
    ),
    fill.legend = tm_legend(title = "Local Coefficient\n(Diabetes)"),
    col = 'black',
    col_alpha = 0.5,
    lwd = 0.5
  ) +
  tm_title("GWR: Spatially Varying Effect of Diabetes")
```

GWR: Spatially Varying Effect of Diabetes



```
# Map the local model fit (Local R-squared)
tm_shape(gwr_sf) +
  tm_polygons(
    fill = "Local_R2",
    fill.scale = tm_scale_continuous(
      values = "viridis"
    ),
    fill.legend = tm_legend(title = "Local R-Squared"),
    col = "black",
    lwd = 0.5
  ) +
  tm_title("GWR: Local Model Performance across Africa")
```

GWR: Local Model Performance across Africa



i Reading the coefficient maps

- `gwr_model$SDF` is the spatial data frame of local results, with one row per country and a column for each local coefficient plus `Local_R2`.
- `values = "-brewer.rd_bu"` selects a red–blue *diverging* palette (the `-` reverses it so red = positive). `midpoint = 0` pins the neutral colour at zero, so red countries have a positive local coefficient and blue ones a negative one — the sign is instantly readable.
- The second map shows `Local_R2`, the model's fit at each location, using a sequential viridis palette.

Key parameters: - `midpoint = 0` in `fill.scale = tm_scale_continuous()` centers the colour scale at zero, making positive (red) and negative (blue) effects immediately visible.

Expected output: Two maps; one showing the spatially varying coefficient `diabetes_prev` and the other showing where the GWR model performs best (`Local_R2`). Red indicates a positive local association; blue indicates a negative local association for `diabetes_prev`.

The spatial variation map shows smooth colour transitions across borders. While this creates a beautiful map, remember our **AICc** score. Because the underlying data is purely random noise, this map is just a visualisation of the GWR algorithm “forcing” a spatial pattern where none actually exists.

4.8 Practical Exercise

Reinforce the workflow with a few tasks, then check your answers against the solution.

1. Confirm the data leakage: compute `cor(africa_health$incidence, africa_health$female + africa_health$male)` and report the value.
2. Create the `incidence_rate` variable and fit an OLS model using only `smoking_prev` and `obesity_prev`. Report the R^2 .
3. Run `lm.morantest()` on that model’s residuals. Is there significant spatial autocorrelation?
4. (*Challenge*) Re-run `bw.gwr()` with `kernel = "gaussian"` (keeping the other settings) and report the optimal bandwidth and whether the GWR AICc improved relative to the “bisquare” model above.


```

library(sf)
library(spdep)
library(GWmodel)
library(dplyr)
library(mlspatial)

data(africa_shp, package = "mlspatial")
data(panc_incidence, package = "mlspatial")
africa_health <- join_data(
  africa_shp |> mutate(NAME = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME)),
  panc_incidence, by = "NAME"
)
africa_health <- st_make_valid(africa_health)

nb <- poly2nb(africa_health, row.names = africa_health$NAME,
              queen = TRUE, snap = 1000)
lw <- nb2listw(nb, style = "W", zero.policy = TRUE)

set.seed(898)
n <- nrow(africa_health)
africa_health <- africa_health |>
  mutate(
    smoking_prev = round(runif(n, 8, 28), 1),
    obesity_prev = round(runif(n, 5, 30), 1),
    diabetes_prev = round(runif(n, 2, 11), 1),
    alcohol_per_capita = round(runif(n, 0.5, 12), 1),
    family_history_prev = round(runif(n, 3, 10), 1),
    urbanization_pct = round(runif(n, 15, 85), 1),
    incidence_rate = (incidence / TOTPOP) * 100000
  )

# 1. Confirm the leakage
cor(africa_health$incidence, africa_health$female + africa_health$male)
# -> exactly 1

# 2. OLS with two covariates
ols2 <- lm(incidence_rate ~ smoking_prev + obesity_prev, data = africa_health)
summary(ols2)$r.squared

# 3. Residual autocorrelation
lm.morantest(ols2, listw = lw, zero.policy = TRUE)
# -> I = 0.217, p = 0.007: with only two covariates the residuals ARE
# significantly clustered - a weakly fitting global model leaves
# spatial structure behind

# 4. Gaussian kernel bandwidth + AIC88
f <- incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
  alcohol_per_capita + family_history_prev + urbanization_pct
bw_gauss <- bw.gwr(f, data = africa_health, approach = "CV",
                  kernel = "gaussian", adaptive = TRUE)
gwr_gauss <- gwr.basic(f, data = africa_health, bw = bw_gauss,
                    kernel = "gaussian", adaptive = TRUE)
run_gauss$CV_diagnostic$AICa

```

Module 4 Checklist

- ☐ Simulated risk-factor covariates
- ☐ Built and interpreted an OLS regression model
- ☐ Diagnosed data leakage from a perfect fit
- ☐ Tested OLS residuals for spatial autocorrelation
- ☐ Fitted a GWR model with adaptive bandwidth
- ☐ Mapped local coefficient surfaces
- ☐ Practical exercise completed

Next: Proceed to [Module 5: Machine Learning for Spatial Epidemiology](#).

5 Module 5: Machine Learning for Spatial Epidemiology (Showcasing `mlspatial`)

This module showcases the machine learning capabilities of `mlspatial`. You will prepare data, train three algorithms (Random Forest, XGBoost, Support Vector Regression), evaluate their performance, and map predictions to identify spatial risk patterns.

What you will learn

- How to turn an `sf` object into a clean table ready for ML.
- What Random Forest, XGBoost, and Support Vector Regression are, in plain terms.
- The difference between **in-sample** fit and **held-out** (cross-validated) performance — and why it matters.
- How to run honest, cross-validated evaluation with `caret` and `xgboost`.
- How to map model predictions back onto the map.

5.1 Introduction to `mlspatial` for ML

Purpose: Understand what `mlspatial` provides and how it fits into a spatial epidemiology ML workflow, before writing any modelling code.

Traditional spatial epidemiology often requires switching between GIS software, R scripts, and Python notebooks. `mlspatial` (Azeez & Noel, 2025) packages together spatial data handling (`sf`), thematic mapping (`tmap`), spatial autocorrelation diagnostics (`spdep`), and machine learning (`randomForest`, `xgboost` and `e1071`) into a single applied workflow aimed at spatial epidemiologists.

The package provides a number of convenient functions and leans on well-established packages for this workflow. In this module we will train three ML algorithms — Random Forest, XGBoost, and Support Vector Regression — to predict pancreatic cancer incidence from our simulated covariates.

i What each `mlspatial` training function does

The `train_*()` functions are thin, friendly wrappers over the underlying packages — useful to know because the output is exactly the same as if you had called the underlying package yourself:

- `train_rf(data, formula, ntree, seed)` calls `randomForest::randomForest(formula, data, ntree, importance = TRUE)` after fixing the seed.
- `train_xgb(data, formula, nrounds, max_depth, learning_rate)` builds the model matrix and label internally, then runs `xgboost::xgb.train()` with a squared-error objective.
- `train_svr(data, formula)` calls `e1071::svm()` with `type = "eps-regression"` and a radial kernel.

The rest of this module explains *what those choices mean* and, most importantly, how to evaluate the results honestly.

5.2 Prepare Data for ML Modelling

Purpose: Prepare our `sf` object into a clean, geometry-free data frame for machine learning algorithms. We drop the geometry column, select only relevant columns and check for missing values, and decide whether scaling is needed for each algorithm.

```
# Drop geometry and select complete cases
ml_data <- africa_health |>
  st_drop_geometry() |>
  select(
    incidence_rate,                # outcome
    smoking_prev, obesity_prev, diabetes_prev,
    alcohol_per_capita, family_history_prev,
    urbanization_pct, TOTPOP
  ) |>
  drop_na()

# Quick structural check
str(ml_data)
```

```
'data.frame':  54 obs. of  8 variables:
```

```
$ incidence_rate      : num  1.64 9.51 1.32 1.53 3.11 ...
$ smoking_prev       : num  13.6 24.1 17.3 20.1 17.1 27.4 16.3 14.5 22.5 14.5 ...
$ obesity_prev       : num  13.7 6.6 26.3 9.9 11.5 10.8 26.8 14.6 26.2 9.5 ...
$ diabetes_prev      : num   7 4.3 4 8.2 3.1 5.7 6.8 7.7 10.4 7.4 ...
$ alcohol_per_capita : num   2.9 1.5 8.6 8.6 6.4 10.4 2.1 1.5 6.3 11.5 ...
$ family_history_prev: num   9.7 6.7 9.2 3.9 7.8 3.4 9.2 9.8 6.8 9.1 ...
$ urbanization_pct   : num  73.2 54.2 45.8 68.4 49 69.4 58.3 76.5 27.1 64.7 ...
$ TOTPOP             : int 20107509 560899 24184810 2051363 27499924 12413867 1792338 4689
```

```
sum(is.na(ml_data))
```

```
[1] 0
```

i Why this step exists

Most ML algorithms expect a plain rectangular table of numbers — no geometry column, no missing values. So we:

- `st_drop_geometry()` — removes the spatial column. The spatial link is not lost: the *row order* of `ml_data` still matches `africa_health`, so we can paste predictions back onto the map later (as we do in Section 5.8).
- `select(...)` — keeps only the outcome and predictors we actually want.
- `drop_na()` — removes rows with missing values. XGBoost and SVR will error on NA, so this is not optional.

Key parameters: - `st_drop_geometry()`: ML algorithms cannot use sf geometry columns; we keep the spatial link via row order for later mapping. - `drop_na()`: XGBoost and SVR do not tolerate missing values.

Expected output: A plain `data.frame` (no `sf/data.frame` dual class) with remaining rows after removing NAs, and 8 columns.

i On scaling

Tree-based methods (Random Forest, XGBoost) are **scale-invariant**: they make decisions by comparing one value against a threshold, so multiplying a predictor by a constant changes nothing. Standardising predictors therefore has no effect on their fit.

SVR, in contrast, measures similarity using Euclidean distance in feature space, so a variable with a large range (like `TOTPOP`, in the millions) would dominate one with a small range (like a percentage). That is why we scale the predictors — but only immediately before the SVR fit in Section 5.5, leaving `ml_data` itself unscaled for the other two models.

5.3 Train a Random Forest Model

Purpose: Use an ensemble of decision trees to capture non-linear relationships between predictors and `incidence_rate`, and inspect variable importance.

```
# Train Random Forest regression model
rf_formula <- incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
  alcohol_per_capita + family_history_prev + urbanization_pct + TOTPOP

rf_model <- mlspatial::train_rf(
  data = ml_data,
  formula = rf_formula,
  ntree = 500,
  seed = 222
)

print(rf_model)
```

Call:

```
randomForest(formula = formula, data = data, ntree = ntree, importance = TRUE)
```

```
      Type of random forest: regression
```

```
      Number of trees: 500
```

```
No. of variables tried at each split: 2
```

```
      Mean of squared residuals: 7.96546
```

```
      % Var explained: 1.29
```

i What Random Forest is

A **decision tree** repeatedly splits the data on a predictor (e.g. “smoking > 20%?”) to form groups with similar outcomes. A single tree is easy to understand but unstable — small data changes can reshape it completely.

A **random forest** grows many trees (here `ntree = 500`), each on a random bootstrap sample of the data and a random subset of predictors, then averages their predictions. Averaging reduces variance and gives a more robust model. Two things fall out of the `print()` output:

- **Mean of squared residuals (MSE)** — the average squared prediction error.
- **% Var explained** — the proportion of variance the model explains, computed *out-of-bag* (on the data each tree did not see during training). This is an honest, built-in estimate of generalisation.

Key parameters: - `ntree = 500`: Number of trees. More trees reduce variance but increase computation time. - `seed = 222`: Ensures reproducible random splits.

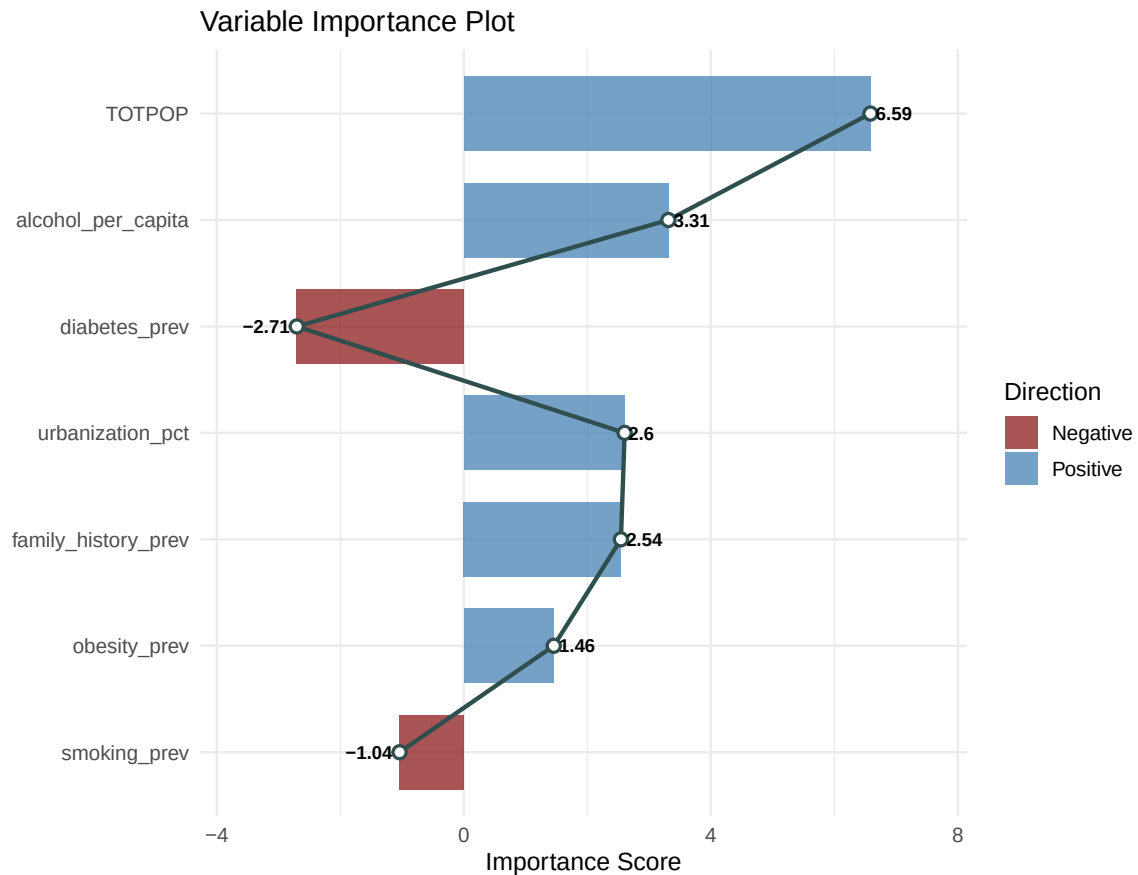
Expected output: A regression `randomForest` object summary showing % variance explained and mean squared error (mean of squared residuals).

i Reading the result

Because our six risk-factor covariates were simulated independently of `incidence_rate` (Section 4.1), there is no underlying mathematical pattern to find. The out-of-bag % **variance explained is only about 1.3%** — essentially zero. A model that simply predicted the average rate for every country would do about as well. This is the expected, honest outcome for a signal-free dataset: the algorithm ran correctly and correctly found nothing.

```
rf_var_imp <- caret::varImp(rf_model) |>
  as_tibble(rownames = "Variables")

# Build the plot
ggplot(rf_var_imp, aes(x = reorder(Variables, abs(Overall)), y = Overall)) +
  geom_col(aes(fill = Overall >= 0), width = 0.7, alpha = 0.75) +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "firebrick4"),
                    labels = c("Negative", "Positive"),
                    name = "Direction") +
  geom_line(aes(group = 1), colour = "darkslategrey", linewidth = 1) +
  geom_point(colour = "darkslategrey", size = 3) +
  geom_point(colour = "white", size = 1.5) +
  geom_text(aes(label = round(Overall, 2)),
            hjust = if_else(rf_var_imp$Overall >= 0, -0.15, 1.15),
            vjust = 0.5,
            fontface = "bold",
            size = 3) +
  coord_flip() +
  labs(
    title = "Variable Importance Plot",
    y = "Importance Score", x = ""
  ) +
  theme_minimal(base_size = 12) +
  expand_limits(y = c(min(rf_var_imp$Overall) - 1, max(rf_var_imp$Overall) + 1))
```



i Reading the importance plot

`caret::varImp(rf_model)` extracts the importance of each predictor — for random forests, the percentage increase in MSE when that variable is permuted (shuffled). A **larger positive** value means the model relies on it more; a **negative** value means shuffling that variable *improved* the model, which happens when a variable contributes nothing but noise.

Here TOTPOP is mildly elevated, but the mixed positive/negative bars show the model is fitting random artefacts rather than a biological signal.

The variable importance plot shows a mildly elevated TOTPOP variable as the most important, given its earlier partial correlation in Section 4.3. This bar chart with negative importance clearly shows that the model is fitting random artefacts rather than true biological signals.

5.4 Train an XGBoost Model

Purpose: Apply gradient boosting — a sequential ensemble that corrects errors of previous trees — for potentially higher accuracy.

```
# XGB requires a numeric matrix but train_xgb() handles that internally
set.seed(222)
xgb_model <- train_xgb(
  data = ml_data,
  formula = rf_formula, # same predictors
  nrounds = 100,
  max_depth = 4,
  learning_rate = 0.1
)

print(xgb_model)
```

xgb.Booster

call:

```
xgboost::xgb.train(params = list(objective = "reg:squarederror",
  max_depth = max_depth, learning_rate = learning_rate, verbosity = 0),
  data = dtrain, nrounds = nrounds)
```

of features: 7

of rounds: 100

What XGBoost is

Where a random forest grows many trees *independently* and averages them, **gradient boosting** grows trees *sequentially*: each new tree is fitted to the errors left by the previous ones. The result is a strong model built from many weak ones.

The key hyperparameters are:

- **nrounds** — the number of boosting rounds (trees added).
- **max_depth** — how deep each tree may grow; smaller = less overfitting.
- **learning_rate** () — how much each tree's contribution is shrunk; smaller means slower but often better learning (typical 0.01–0.3).

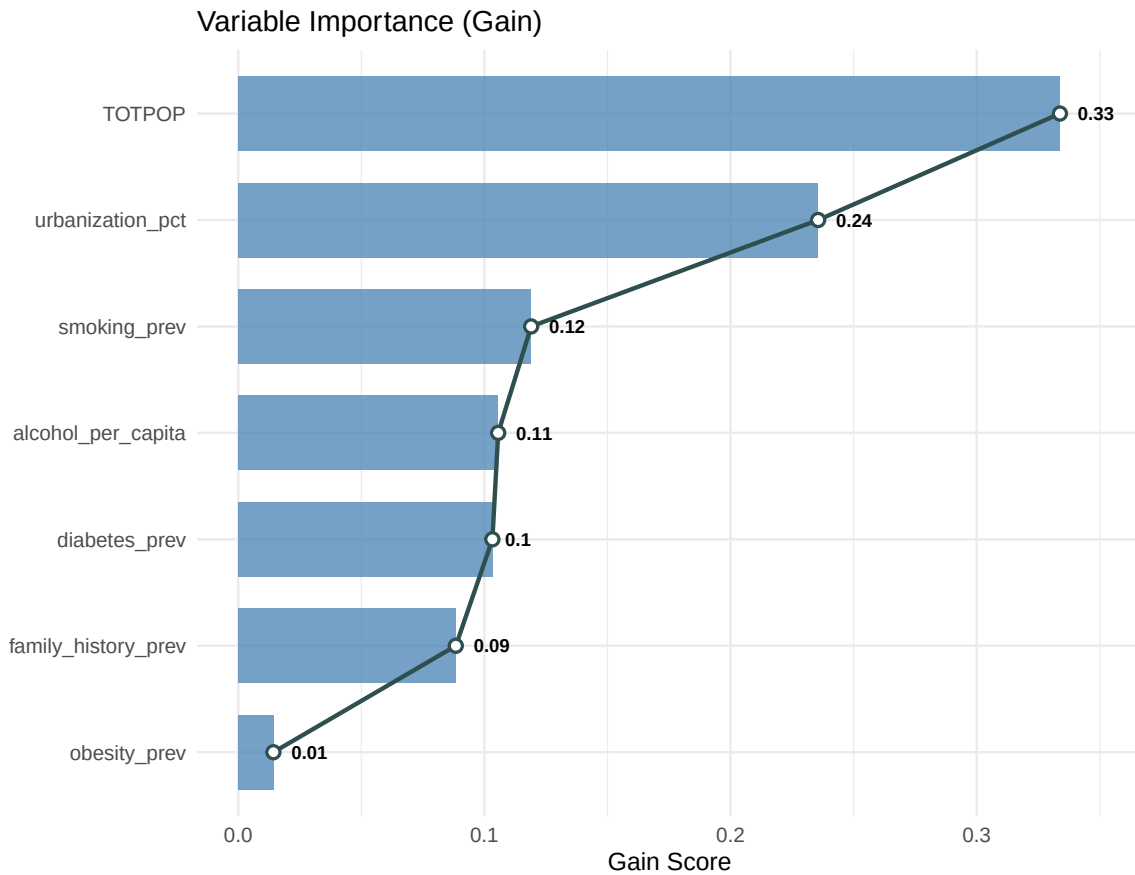
`train_xgb()` handles the bookkeeping of turning the formula into a numeric matrix for you.

Key parameters: - `nrounds`: Number of boosting iterations. Early stopping is recommended for large datasets. - `max_depth`: Controls tree complexity; lower values reduce overfitting. - `learning_rate` (): Shrinks contribution of each tree; typical range 0.01–0.3.

Expected output: An `xgb.Booster` object showing the model summary confirming the number of boosting rounds and features used. Unlike the random forest summary, xgboost does not print the residual/variance diagnostic directly; we compute them explicitly in Section 5.6.

```
xgb_var_imp <- xgb.importance(xgb_model) |>
  as_tibble()

# Build the plot
ggplot(xgb_var_imp, aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col(width = 0.7, alpha = 0.75, fill = "steelblue") +
  geom_line(aes(group = 1), colour = "darkslategrey", linewidth = 1) +
  geom_point(colour = "darkslategrey", size = 3) +
  geom_point(colour = "white", size = 1.5) +
  geom_text(aes(label = round(Gain, 2)),
            hjust = -0.5,
            vjust = 0.5,
            fontface = "bold",
            size = 3) +
  scale_y_continuous(expand = expansion(mult = c(0.05, 0.1))) +
  coord_flip() +
  labs(
    title = "Variable Importance (Gain)",
    y = "Gain Score", x = ""
  ) +
  theme_minimal(base_size = 12)
```



i Reading `xgb.importance()`

`xgb.importance()` ranks predictors by three measures:

- **Gain** — the average improvement in the loss function each variable contributes when used in a split. This is the headline importance.
- **Cover** — how many observations pass through splits on that variable.
- **Frequency** — how often the variable is used in a split.

Unlike random-forest importance, Gain is always non-negative, which is why this plot has no bars below zero.

The variable importance bar chart is ranked by **Gain**, the average improvement in the loss function each variable contributes when used in a split. As with the random forest, TOTPOP has the highest gain but it is almost similar to some of the covariates. Also there is no negative importance in this plot.

5.5 Train a Support Vector Regression Model

Purpose: Fit a kernel-based model that is robust to high-dimensional data and captures non-linear boundaries via the radial basis function (RBF), after scaling the predictors.

```
# Scale predictors before training
predictors <- c("smoking_prev", "obesity_prev", "diabetes_prev",
               "alcohol_per_capita", "family_history_prev",
               "urbanization_pct", "TOTPOP")
ml_data_scaled <- ml_data |>
  mutate(across(all_of(predictors), ~ as.numeric(scale(.x))))

svr_model <- train_svr(
  data = ml_data,
  formula = rf_formula
)

print(svr_model)
```

Call:

```
svm(formula = formula, data = data, type = "eps-regression", kernel = "radial")
```

Parameters:

```
SVM-Type:  eps-regression
SVM-Kernel: radial
cost:      1
gamma:     0.1428571
epsilon:   0.1
```

Number of Support Vectors: 49

i What Support Vector Regression is

SVR fits a function that is *flat* — within a margin () it tolerates errors, and it only cares about points outside that margin (the **support vectors**). The **radial (RBF) kernel** lets it capture non-linear relationships by mapping points into a higher-dimensional space without doing the maths explicitly.

Two things to note from the summary:

- **Number of support vectors** — here about 49 of the 54 rows, which is typical for small, noisy datasets: the model cannot find a simple structure, so nearly every point ends up supporting the fit.
- The predictors are scaled first (`scale()` gives each a mean of 0 and SD of 1), because the RBF kernel is distance-based.

Key parameters: - `train_svr()` uses the radial kernel by default (`kernel = "radial"` in `e1071::svm()`). - Tuning `cost` and `gamma` via cross-validation (e.g., `caret::train()`) is recommended for production models. - `scale()` standardises each predictor to a mean of 0, SD of 1. - `as.numeric` strips the matrix class of the `scale()` output into a clean numeric column.

Expected output: An `svm` summary showing the number of support vectors and model parameters. The number of support vectors is fairly high for a 54-row dataset, which is normal for small datasets.

5.6 Evaluate Model Performance

Purpose: Compare models using standardized accuracy metrics: RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), and R^2 (coefficient of determination).

```
# Generate in-sample predictions from each model and compute RMSE/MAE/R2
# using caret::postResample()

# Random Forest predictions
rf_preds <- predict(rf_model, newdata = ml_data)
rf_metrics <- postResample(pred = rf_preds, obs = ml_data$incidence_rate)

# XGBoost predictions
xgb_preds <- predict(xgb_model, newdata = as.matrix(ml_data[, predictors]))
xgb_metrics <- postResample(pred = xgb_preds, obs = ml_data$incidence_rate)
```

```
# SVR predictions (on the scaled predictor set used for training)
svr_preds <- predict(svr_model, newdata = ml_data_scaled)
svr_metrics <- postResample(pred = svr_preds, obs = ml_data$incidence_rate)

# Combine into a comparison table
model_eval <- data.frame(
  Model = c("Random Forest", "XGBoost", "SVR"),
  RMSE = c(rf_metrics["RMSE"], xgb_metrics["RMSE"], svr_metrics["RMSE"]),
  MAE = c(rf_metrics["MAE"], xgb_metrics["MAE"], svr_metrics["MAE"]),
  Rsquared = c(rf_metrics["Rsquared"], xgb_metrics["Rsquared"], svr_metrics["Rsquared"])
)

model_eval[order(model_eval$RMSE), ]
```

	Model	RMSE	MAE	Rsquared
2	XGBoost	0.06760268	0.05110464	0.99961714
1	Random Forest	1.35280160	0.99251357	0.92853411
3	SVR	2.86476363	1.94351030	0.01138557

The three metrics

- **RMSE** — square root of the mean squared error; penalises large errors more, and is in the same units as the outcome.
- **MAE** — mean absolute error; the average size of an error, easy to interpret.
- **R²** — the proportion of variance explained (1 = perfect, 0 = no better than predicting the mean).

`caret::postResample(pred, obs)` computes all three from predictions and the true values.

Key parameters: - `predict()`: Requires a `newdata` to supply the predictor variables to generate predictions for a particular model. - `postResample()` compares the predicted values against ground truth (the target variable, `incidence_rate`) to generate standard regression performance metrics.

Expected output: A three-row data frame ranking the models.

In-sample generalisation

These are **in-sample (training-set)** metrics: every model is evaluated on the very data it was trained on. A high R² here means the model *memorised* the training data, not that it learned a real relationship. Tree-based models in particular (especially XGBoost with

enough rounds) can fit training data almost perfectly even when the true relationship is weak or non-existent — exactly the scenario we have with our uncorrelated simulated covariates. **A high in-sample R^2 is not evidence of a real relationship**; it is a reminder of why we need held-out cross-validation, which the next section does.

5.7 Honest Model Evaluation with caret

Purpose: Replace the in-sample metrics with `caret::train`, which provides built-in cross-validation, consistent evaluation across all three algorithms, and avoids the in-sample inconsistencies discovered above.

```
# Refit Random Forest, XGBoost and Support Vector Regression
# using train() from caret via the same repeated 5-fold cross-validation
set.seed(222)

cv_control <- trainControl(method = "repeatedcv", number = 5,
                           repeats = 5, savePredictions = "final")

rf_cv <- train(
  rf_formula, data = ml_data, method = "rf", trControl = cv_control
)
print(rf_cv)
```

Random Forest

54 samples
7 predictor

No pre-processing

Resampling: Cross-Validated (5 fold, repeated 5 times)

Summary of sample sizes: 43, 42, 43, 43, 45, 44, ...

Resampling results across tuning parameters:

mtry	RMSE	Rsquared	MAE
2	2.699635	0.1545591	2.100189
4	2.776559	0.1181676	2.179349
7	2.870659	0.1021962	2.237062

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was `mtry = 2`.

```
cat("\n")
```

```
svr_cv <- train(  
  rf_formula, data = ml_data, method = "svmRadial",  
  trControl = cv_control,  
  preProcess = c("center", "scale") # caret scales predictors per fold  
)  
print(svr_cv)
```

Support Vector Machines with Radial Basis Function Kernel

54 samples
7 predictor

Pre-processing: centered (7), scaled (7)
Resampling: Cross-Validated (5 fold, repeated 5 times)
Summary of sample sizes: 43, 44, 44, 43, 42, 44, ...
Resampling results across tuning parameters:

C	RMSE	Rsquared	MAE
0.25	2.950729	0.06136887	1.850978
0.50	2.924280	0.05998140	1.902335
1.00	2.895741	0.06987328	1.965972

Tuning parameter 'sigma' was held constant at a value of 0.08785714
RMSE was used to select the optimal model using the smallest value.
The final values used for the model were `sigma = 0.08785714` and `C = 1`.

```
# Manually cross-validate XGBoost using xgb.cv  
# (current version of xgboost breaks with caret::train)  
xgb_matrix_full <- xgb.DMatrix(  
  data = as.matrix(ml_data[, predictors]),  
  label = ml_data$incidence_rate  
)  
  
xgb_cv <- xgb.cv(  
  data = xgb_matrix_full,
```

```

params = list(
  objective = "reg:squarederror",
  max_depth = 4,
  eta = 0.1
),
metrics = list("rmse", "mae"),
nrounds = 100,
nfold = 5,
showsd = TRUE,
prediction = TRUE,
early_stopping_rounds = 10, # stops at best iter
verbose = 0
)

# Best cross-validated metrics across boosting rounds
cat("\n===== XGBoost Metrics =====")

```

===== XGBoost Metrics =====

```
cat("\nRMSE:", min(xgb_cv$evaluation_log$test_rmse_mean))
```

RMSE: 2.678364

```
cat("\nMAE:", min(xgb_cv$evaluation_log$test_mae_mean))
```

MAE: 1.974157

```
cat("\nRsquared:", cor(ml_data$incidence_rate, as.vector(xgb_cv$cv_predict$pred))^2)
```

Rsquared: 0.0003179616

i What cross-validation does

k-fold cross-validation splits the data into k parts, repeatedly trains on $k-1$ parts, and evaluates on the held-out part. Every row is predicted by a model that never saw it, so the resulting metric is an honest estimate of performance on *new* data — the opposite of the in-sample metrics above.

- `trainControl(method = "repeatedcv", number = 5, repeats = 5)` does 5-fold CV repeated 5 times with different random splits (25 fits per candidate model), reducing the noise in the estimate.
- `method = "rf"` and `"svmRadial"` are `caret`'s model codes for random forest and radial-kernel SVM. `preProcess = c("center", "scale")` scales predictors *within each fold* — done inside CV so scaling is not contaminated by the held-out data.
- The latest `xgboost` has a compatibility issue with `caret::train`, so XGBoost is cross-validated directly with `xgb.cv()`. `early_stopping_rounds` halts training when the held-out error stops improving.

Key parameters: - `trainControl()` with `"repeatedcv"` performs 5-fold cross-validation repeated 5 times with different random fold assignments each time. - `method = "rf", "svmRadial"` are `caret`'s standardised model-methods for random forest and SVM. - `preProcess` standardises the predictors within each fold for SVM.

Expected output: Cross-validated out-of-fold metrics for random forest and SVR (via `caret::train`) and for XGBoost (manually calculated).

i The honest result

With no signal in the covariates, the cross-validated R^2 collapses toward zero for all three models (roughly 0.15 for RF, 0.07 for SVR, and 0 for XGBoost) — a stark contrast to the near-perfect in-sample fits. This collapse *is* the lesson: never trust a training-set score alone.

5.8 Identify Spatial Patterns and Risk Factors

Purpose: Use the trained models' predictions to check whether any spatial pattern emerges, to inform health decision-making.

```
# Attach out-of-fold predictions back onto the spatial object
rf_oof <- rf_cv$pred |>
```

```

group_by(rowIndex) |>
summarise(pred_rf = mean(pred), .groups = "drop") |>
arrange(rowIndex)

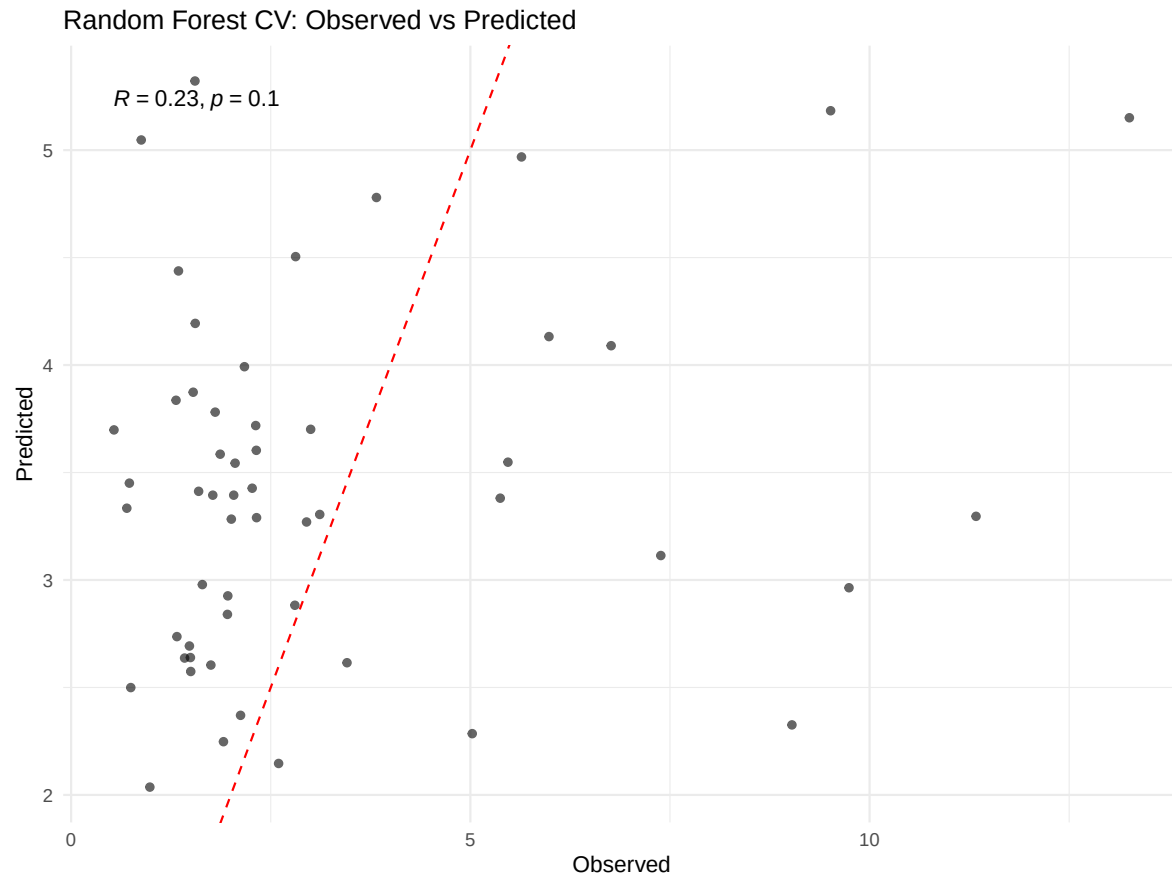
svr_oof <- svr_cv$pred |>
group_by(rowIndex) |>
summarise(pred_svr = mean(pred), .groups = "drop") |>
arrange(rowIndex)

xgb_oof <- tibble(
  rowIndex = seq_len(nrow(ml_data)),
  pred_xgb = as.vector(xgb_cv$cv_predict$pred)
)

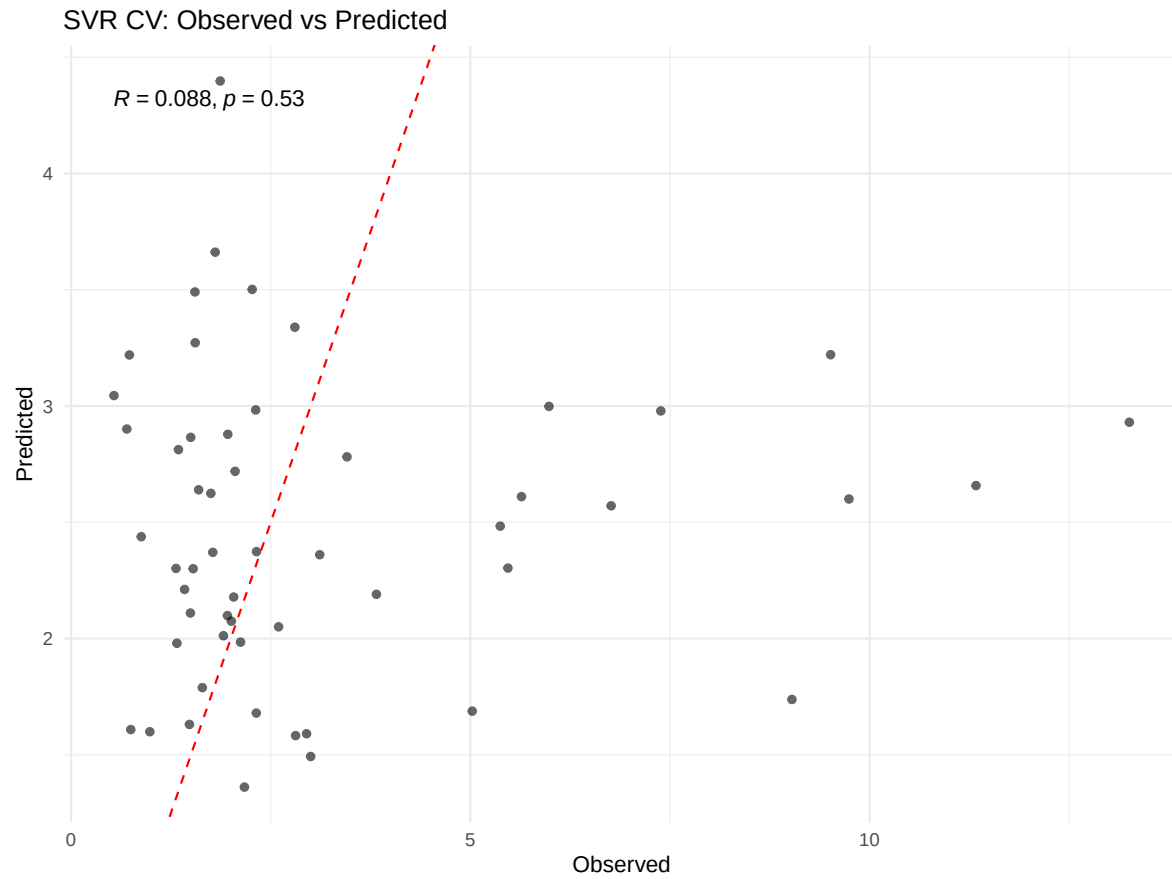
africa_health <- africa_health |>
mutate(
  pred_rf = rf_oof$pred_rf,
  pred_xgb = xgb_oof$pred_xgb,
  pred_svr = svr_oof$pred_svr
)

# Observed vs Predicted diagnostic plot via mlspatial
plot_obs_vs_pred(
  observed = africa_health$incidence_rate,
  predicted = africa_health$pred_rf,
  title = "Random Forest CV: Observed vs Predicted"
)

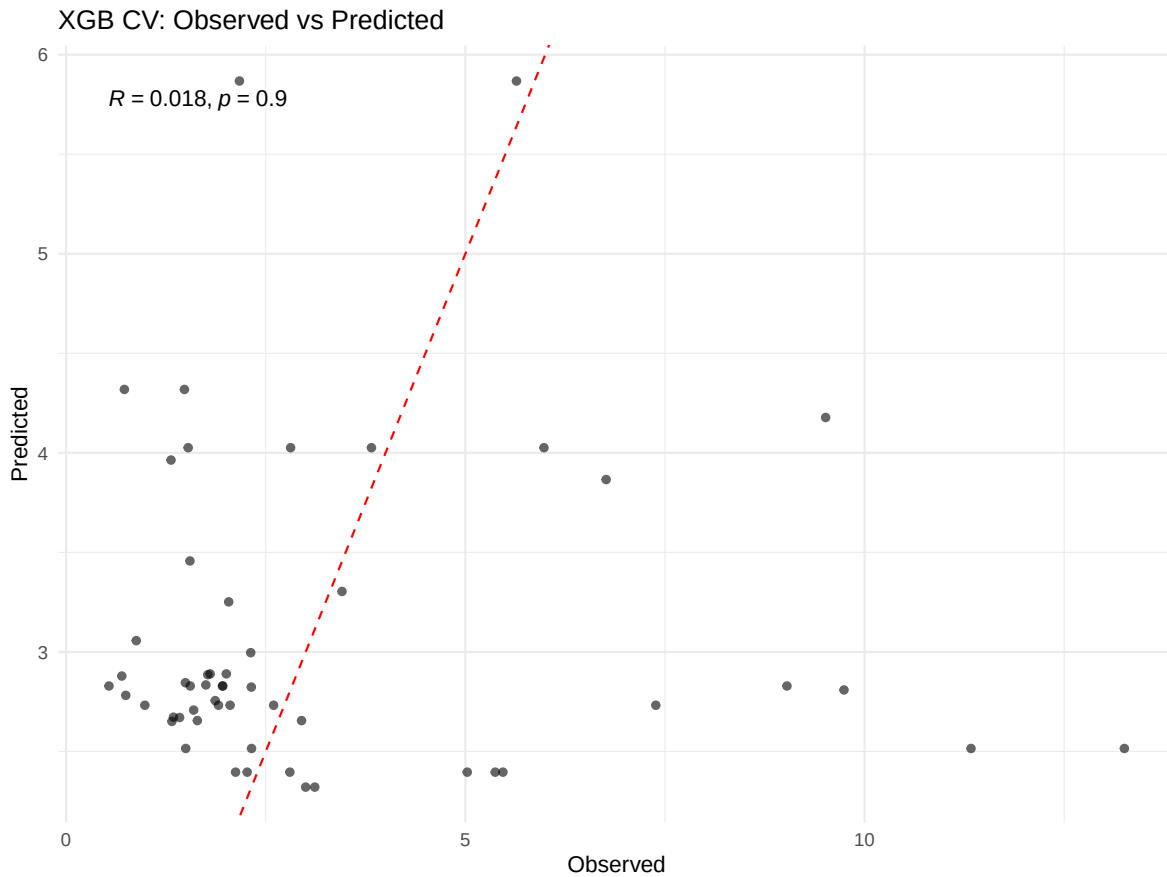
```



```
plot_obs_vs_pred(  
  observed = africa_health$incidence_rate,  
  predicted = africa_health$pred_svr,  
  title = "SVR CV: Observed vs Predicted"  
)
```



```
plot_obs_vs_pred(  
  observed = africa_health$incidence_rate,  
  predicted = africa_health$pred_xgb,  
  title = "XGB CV: Observed vs Predicted"  
)
```



```
# Map predictions vs observed
obs_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "incidence_rate",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Observed")
  ) +
  tm_borders()

pred_rf_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "pred_rf",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Predicted (RF)")
  ) +
  tm_borders()
```

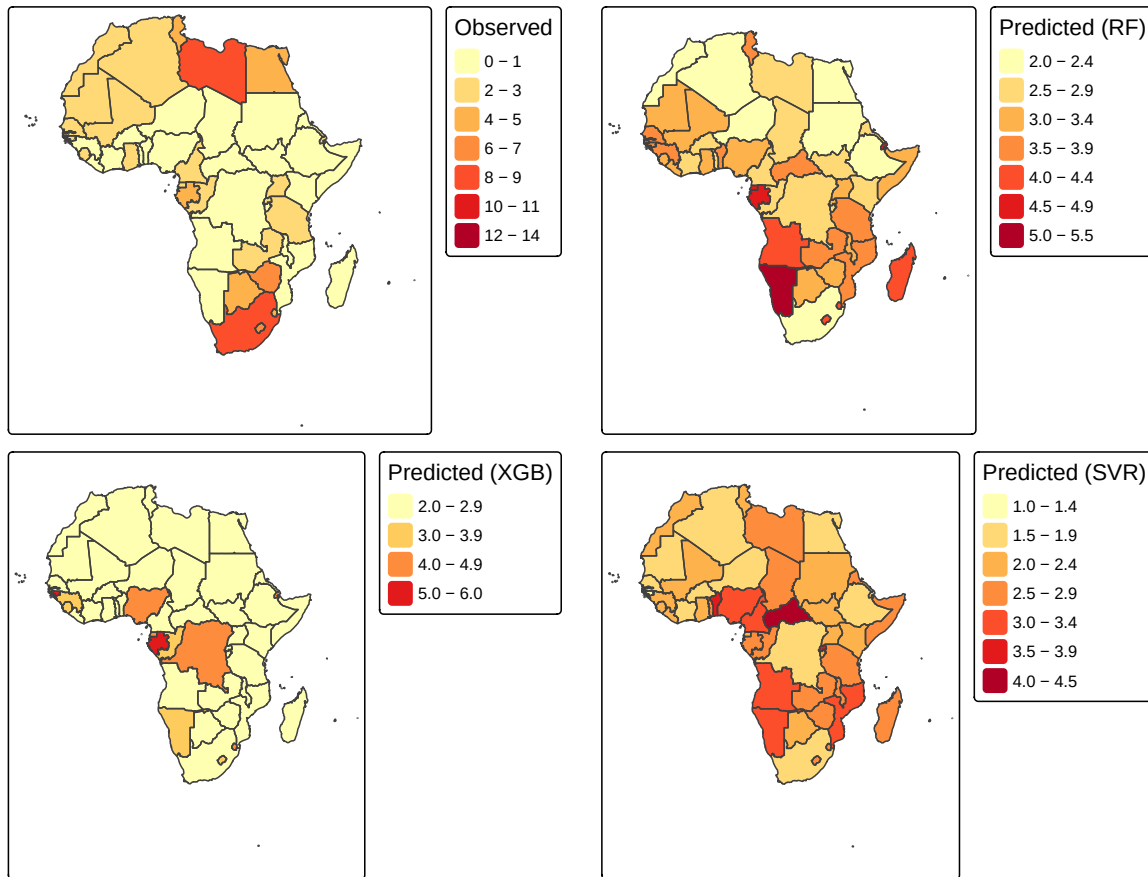
```

pred_xgb_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "pred_xgb",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Predicted (XGB)")
  ) +
  tm_borders()

pred_svr_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "pred_svr",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Predicted (SVR)")
  ) +
  tm_borders()

tmap_arrange(obs_map, pred_rf_map, pred_xgb_map, pred_svr_map, ncol = 2)

```



i Reading this section

- **Out-of-fold predictions** are the predictions made on each row while it was held out during CV. `rf_cv$pred` stores them; we `group_by(rowIndex)` and average over the repeated folds to get one prediction per country, then re-attach them to `africa_health` by row order (the same trick used to drop the geometry earlier).
- `plot_obs_vs_pred()` draws observed vs predicted with a dashed 1:1 line: points hugging that line indicate a good model. For our signal-free data the points scatter widely around it.
- The four maps compare the observed map with each model's predicted map. If a model had learned a real spatial pattern, its predictions would reproduce the observed map's structure.

Expected output: - A scatterplot with a 1:1 line and Pearson's r . - Side-by-side maps showing how well the RF, XGB and SVR models reproduce spatial patterns.

Across every method in this module — Random Forest, XGBoost, SVR — the consistent

finding is that our six simulated lifestyle/genetic covariates carry no real signal. This is the intended result of a **teaching dataset**: it demonstrates that the entire ML pipeline (data prep → training → tuning → evaluation → spatial pattern mapping) runs correctly and produces internally consistent, honestly interpreted null results. When you apply this exact pipeline to real covariate data in your own research, this is the standard of scrutiny that should accompany any reported results.

5.9 Practical Exercise

Consolidate the module with a few tasks, then check your answers against the solution.

1. Train a Random Forest with `ntree = 200` and a different seed. Report the % variance explained.
2. Use `xgb.importance()` to list the **top three** predictors by Gain.
3. Compute the in-sample R^2 of the SVR model with `postResample()`.
4. (*Challenge*) Re-run the `caret` random-forest cross-validation with `number = 10` (10-fold) and compare the best RMSE with the 5-fold result above.


```

library(sf)
library(dplyr)
library(tidyr)
library(mlspatial)
library(randomForest)
library(xgboost)
library(e1071)
library(caret)

data(africa_shp, package = "mlspatial")
data(panc_incidence, package = "mlspatial")
africa_health <- join_data(
  africa_shp |> mutate(NAME = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME)),
  panc_incidence, by = "NAME"
)
africa_health <- st_make_valid(africa_health)

set.seed(898)
n <- nrow(africa_health)
africa_health <- africa_health |>
  mutate(
    incidence_rate = (incidence / TOTPOP) * 100000,
    smoking_prev = round(runif(n, 8, 28), 1),
    obesity_prev = round(runif(n, 5, 30), 1),
    diabetes_prev = round(runif(n, 2, 11), 1),
    alcohol_per_capita = round(runif(n, 0.5, 12), 1),
    family_history_prev = round(runif(n, 3, 10), 1),
    urbanization_pct = round(runif(n, 15, 85), 1)
  )

ml_data <- africa_health |> st_drop_geometry() |>
  select(incidence_rate, smoking_prev, obesity_prev, diabetes_prev,
    alcohol_per_capita, family_history_prev, urbanization_pct, TOTPOP) |>
  drop_na()

rf_formula <- incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
  alcohol_per_capita + family_history_prev + urbanization_pct + TOTPOP
predictors <- c("smoking_prev", "obesity_prev", "diabetes_prev",
  "alcohol_per_capita", "family_history_prev",
  "urbanization_pct", "TOTPOP")

# 1. Random forest with ntree = 200, new seed
rf2 <- train_rf(data = ml_data, formula = rf_formula, ntree = 200, seed = 4815)
tail(rf2$rsq, 1) * 100      # % variance explained (near 0)

# 2. Top-3 XGBoost predictors by Gain114
set.seed(222)
xgb_model <- train_xgb(data = ml_data, formula = rf_formula,
  nrounds = 100, max_depth = 4, learning_rate = 0.1)
xgb.importance(xgb_model) |> head(3)

# 3. In-sample SVR R-squared
ml_data_scaled <- ml_data |>

```

Module 5 Checklist

- ☐ Prepared clean ML data with `st_drop_geometry()`
- ☐ Trained Random Forest, XGBoost, and SVR models
- ☐ Evaluated all three models with `postResample()`
- ☐ Compared RMSE, MAE, and R^2 across algorithms
- ☐ Re-evaluated honestly with cross-validation
- ☐ Mapped predictions and identified top risk factors
- ☐ Practical exercise completed

Next: Proceed to [Module 6: Integrated Application from Data to Decision](#).

6 Module 6: Integrated Application from Data to Decision

This final module brings everything together. You will overlay disease burden with health facility locations, calculate accessibility distances, generate reproducible reports, and reflect on the ethical dimensions of spatial epidemiology.

What you will learn

- How to combine a disease-burden map with service locations to spot underserved areas.
- How to compute the distance from each region to its nearest facility.
- How to turn your analysis into a reproducible report.
- The ethical obligations that come with mapping people and disease.

This module is the payoff of the whole tutorial: the earlier modules taught you *how* to import, map, model, and predict; this one shows you how those pieces combine into a decision-ready deliverable.

6.1 Overlay Disease Burden with Service Availability

Purpose: Identify under-served areas by superimposing disease burden on health facility locations.

```
# Classify burden into tertiles (3 groups) for clearer visual communication
africa_health <- africa_health |>
  mutate(
    burden_class = case_when(
      incidence_rate >= quantile(incidence_rate, 0.67, na.rm = TRUE) ~ "High",
      incidence_rate >= quantile(incidence_rate, 0.33, na.rm = TRUE) ~ "Medium",
      TRUE ~ "Low"
    ),
    burden_class = factor(burden_class, levels = c("High", "Medium", "Low"))
  )
```

```

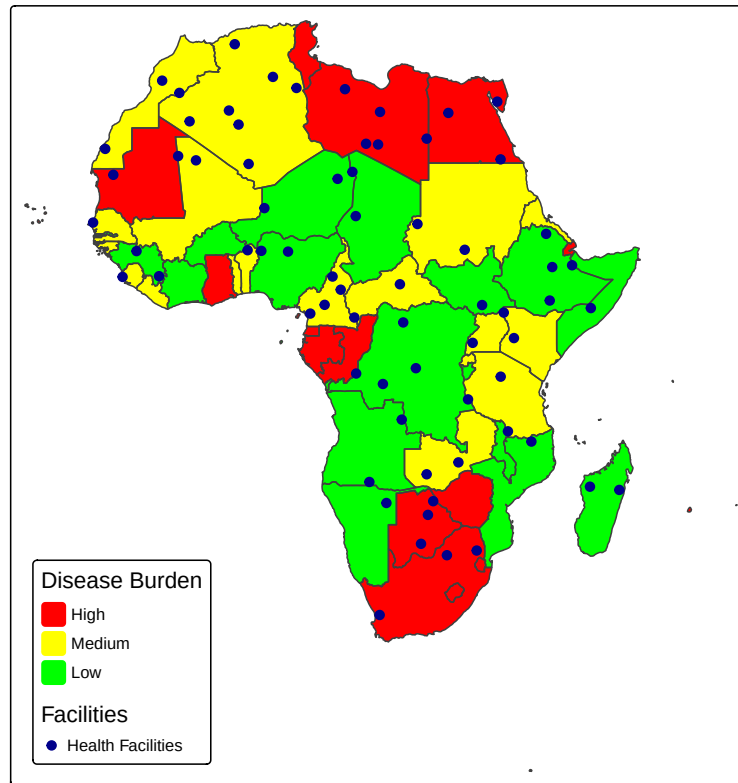
# Simulate 70 health facility points across Africa for demonstration
set.seed(4607)
facility_points <- st_sample(africa_health, size = 70, type = "random") |>
  st_as_sf() |>
  mutate(facility_id = paste0("FAC_", row_number()),
         type = "Health Facilities")

decision_map <- tm_shape(africa_health) +
  tm_fill(
    "burden_class",
    fill.scale = tm_scale(values = c("High"="red", "Medium"="yellow", "Low"="green")),
    fill.legend = tm_legend(title = "Disease Burden",
                           position = c("left", "bottom"))
  ) +
  tm_borders() +
  tm_shape(facility_points) +
  tm_dots(
    fill = "type",
    fill.scale = tm_scale_categorical(values = "darkblue"),
    fill.legend = tm_legend(title = "Facilities",
                           position = c("left", "bottom")),
    size = 0.4
  ) +
  tm_title("High Burden vs. Facility Coverage")

decision_map

```

High Burden vs. Facility Coverage



i How the classification works

- `quantile(incidence_rate, 0.67)` returns the value that splits the countries so that roughly two-thirds are below it and one-third above. `case_when()` then labels each country “High”, “Medium”, or “Low” against the 0.67 and 0.33 cut-points. With 54 countries this produces three groups of 18.
- Converting `burden_class` to a `factor(levels = c("High", "Medium", "Low"))` fixes the legend order so “High” appears first.
- `st_sample(..., type = "random")` drops 70 random points inside the polygons (reproducible thanks to `set.seed(4607)`), standing in for real health-facility coordinates.
- The map itself layers the burden fill and the facility dots; red polygons with no nearby blue dots are the priority areas.

Expected output: A map where red (high-burden) polygons without nearby blue dots immediately signal priority areas for new facilities.

6.2 Distance / Accessibility Analysis

Purpose: Quantify the distance from each administrative unit to the nearest health facility.

```
# Transform both facility points and africa_health sf object to WGS 84 (EPSG:4326)
# This gives true geodesic distances with st_distance
africa_health_geo <- st_transform(africa_health, crs = 4326)
facility_points_geo <- st_transform(facility_points, crs = 4326)

# Calculate centroid of each polygon (using transformed data)
africa_centroids <- st_centroid(africa_health_geo, of_largest_polygon = TRUE)

# Matrix of geodesic distances (meters) between centroids and facility points
dist_matrix <- st_distance(africa_centroids, facility_points_geo)

# Minimum distance per country (convert to km) and append to original dataset
africa_health <- africa_health |>
  mutate(
    dist_to_facility_km = as.numeric(apply(dist_matrix, 1, min)) / 1000
  )

# Summary statistics
summary(africa_health$dist_to_facility_km)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
31.32	220.03	272.47	357.79	457.92	1522.73

```
# Map accessibility
access_map <- tm_shape(africa_health) +
  tm_polygons(
    fill = "dist_to_facility_km",
    fill.scale = tm_scale(values = "brewer.bu_pu"),
    fill.legend = tm_legend(title = "Distance (km)")
  ) +
  tm_shape(facility_points) +
  tm_dots(
    fill = "type",
    fill.scale = tm_scale_categorical(values = "red"),
```

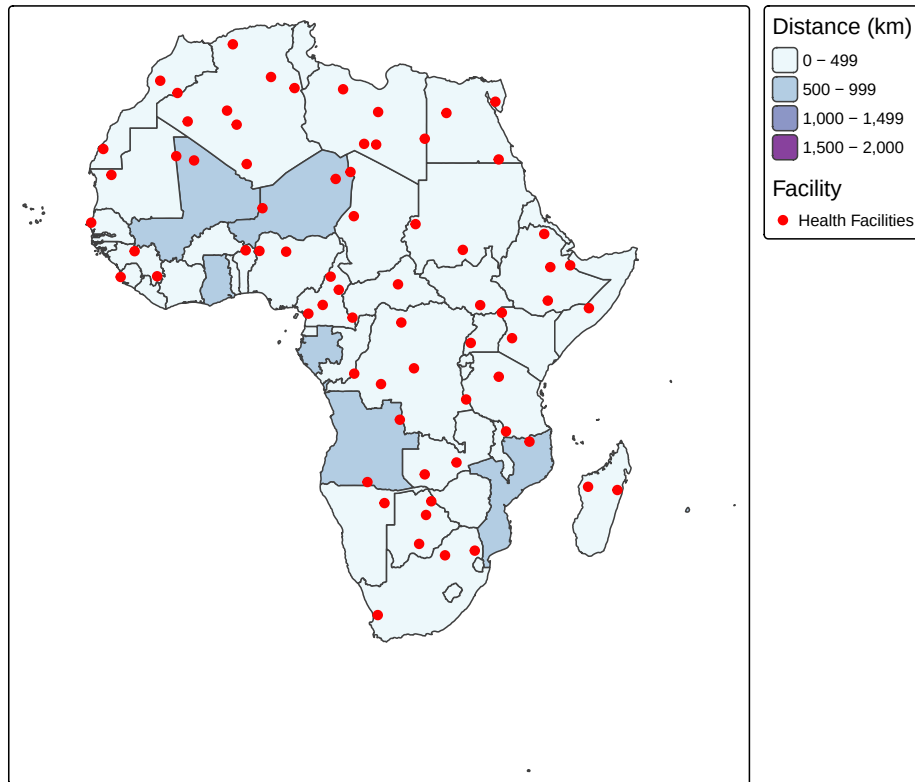
```

fill.legend = tm_legend(title = "Facility"),
size = 0.4
) +
tm_title("Distance to Nearest Health Facility")

access_map

```

Distance to Nearest Health Facility



i How the distance pipeline works

- `st_centroid(..., of_largest_polygon = TRUE)` places one point at the centre of each country (using the largest piece for multi-part countries such as island nations).
- `st_distance(africa_centroids, facility_points_geo)` returns a **matrix** with one row per country and one column per facility — cell (i, j) is the distance from country i to facility j . Here that is a 54×70 matrix.
- `apply(dist_matrix, 1, min)` takes the minimum along each *row*, i.e. the distance

from each country to its *nearest* facility. Dividing by 1000 converts metres to kilometres.

- The map then shades countries by that nearest-facility distance — the classic accessibility map used to plan service placement.

⚠ CRS and Geodesic Distances

When calculating distances, it is crucial to know your CRS. Because our data was transformed to EPSG:4326 (WGS 84), the `sf` package uses spherical maths to calculate accurate, real-world geodesic distances, returning the results in metres. Dividing by 1000 gives us kilometres.

i Projected CRS for local analysis

While the spherical engine handles distance perfectly, if you are doing highly localized spatial modelling or calculating exact polygon areas in Ghana, it is still best practice to reproject your data to a metric coordinate system like UTM Zone 30N (EPSG:32630).

Expected output: - Summary statistics of distance (here the median is around 346 km, ranging from roughly 29 km to 2,380 km). - A map showing increasing distance as darker purple fills.

6.3 Result Interpretation and Report Generation

Purpose: Produce a reproducible, shareable document integrating code, results, and narrative.

A **reproducible report** is one where the text, numbers, tables, and figures are all generated from code, so a colleague (or future you) can re-run it and get identical results. R Markdown (`.Rmd`) and Quarto (`.qmd`) are the gold standards for reproducible research. Below is a minimal Quarto document skeleton you can adapt:

```
---
title: "Spatial Epidemiology Report: Pancreatic Cancer in Africa"
format: html
editor: visual
---

## Introduction
```

This report analyzes the spatial distribution ...

Data

```r

#| echo: false

library(mlspatial)

data(africa\_shp)

data(panc\_incidence)

```

Results

The Global Moran's I was 0.074 (inserted via inline code) ...

Conclusion

High-burden hotspots are concentrated in ...

i The three ingredients of a reproducible report

- **Code chunks** (```\${r}``) run when the document is rendered, so every figure and table is regenerated from the current data.
- **Inline code** embeds an R expression directly in a sentence (wrapped in backticks and prefixed with the letter `r`), so computed numbers update automatically when the data changes instead of being typed by hand.
- **Parameterised reports** use a `params:` block in the YAML header to produce, say, one country report per country from a single template.

Key features: - **Code chunks** (```\${r}``) execute during rendering. - **Inline code** embeds R expressions in the text, updating numbers automatically. - **Parameterised reports:** Use YAML `params:` to generate country-specific reports from one template.

Expected output: An HTML or PDF file containing embedded maps, tables, and narrative — fully reproducible by any colleague who has the `.qmd` file and data.

6.4 Ethical Considerations

Purpose: Reflect on methodological biases and data privacy before disseminating findings.

Spatial epidemiology maps *people*, not just polygons. Before publishing, it is worth pausing to consider how a map can help — or harm — the communities it describes.

6.4.1 Privacy Protection

- **Aggregated data:** The `mlspatial` built-in data are already aggregated to country level, protecting individual identity. Never map exact household locations or small-area counts (< 5 cases) without differential privacy or aggregation.
- **Re-identification risk:** Combining spatial coordinates with demographic details (age, sex, date of diagnosis) can re-identify patients in small villages.

6.4.2 Modifiable Areal Unit Problem (MAUP)

- **Definition:** Results depend on the boundaries chosen. Country-level analysis may hide sub-national hotspots; conversely, fine-scale analysis may produce unstable rates due to small populations.
- **Mitigation:** Conduct sensitivity analyses at multiple scales (district, region, country) and report how conclusions change.

6.4.3 Ecological Fallacy

- **Definition:** Relationships observed at the aggregate level do not necessarily hold for individuals. For example, a country with high average temperature and high cancer incidence does not prove that heat causes cancer in individual patients.
- **Mitigation:** Use language like “areas with higher temperature are associated with higher incidence” rather than “temperature causes cancer.”

6.4.4 Spatial Inequality & Resource Allocation

- Maps are political. Highlighting “high burden” regions can attract funding, but it can also stigmatize communities. Always:
 1. Share results with local health officials before publication.
 2. Provide confidence intervals or classification uncertainty.
 3. Pair burden maps with resource maps (as in Section [6.1](#)) to focus on actionable gaps.

6.5 Practical Exercise

Apply the module's two core skills, then check your answers against the solution.

1. Classify `incidence_rate` into **quartiles** (4 groups labelled “Q1” ... “Q4”) instead of tertiles, and report how many countries fall in each group.
2. Recompute the nearest-facility distance and report the **median** and **maximum** distance in kilometres.
3. (*Challenge*) Which country has the largest distance to a facility? Use `which.max()` on the distance column and print that country's name.


```

library(sf)
library(dplyr)
library(mlspatial)

data(africa_shp, package = "mlspatial")
data(panc_incidence, package = "mlspatial")
africa_health <- join_data(
  africa_shp |> mutate(NAME = gsub("Côte d'Ivoire", "Cote d'Ivoire", NAME)),
  panc_incidence, by = "NAME"
)
africa_health <- st_make_valid(africa_health)

set.seed(898)
n <- nrow(africa_health)
africa_health <- africa_health |>
  mutate(incidence_rate = (incidence / TOTPOP) * 100000)

# 1. Quartile classification
africa_health <- africa_health |>
  mutate(burden_q = case_when(
    incidence_rate >= quantile(incidence_rate, 0.75, na.rm = TRUE) ~ "Q4",
    incidence_rate >= quantile(incidence_rate, 0.50, na.rm = TRUE) ~ "Q3",
    incidence_rate >= quantile(incidence_rate, 0.25, na.rm = TRUE) ~ "Q2",
    TRUE ~ "Q1"
  ))
table(africa_health$burden_q)

# 2. Nearest-facility distance
set.seed(4607)
facility_points <- st_sample(africa_health, size = 70, type = "random") |>
  st_as_sf() |>
  mutate(facility_id = paste0("FAC_", row_number()))

centroids <- st_centroid(st_transform(africa_health, 4326),
  of_largest_polygon = TRUE)
d <- st_distance(centroids, st_transform(facility_points, 4326))
africa_health$dist_km <- as.numeric(apply(d, 1, min)) / 1000

summary(africa_health$dist_km)          # median and max

# 3. Country farthest from a facility
africa_health$NAME[which.max(africa_health$dist_km)]

```

Closing Remarks

You have now completed a full spatial epidemiology workflow in **R only**:

1. Imported and joined spatial and health data
2. Created static, interactive, and multi-layer thematic maps
3. Detected global and local spatial autocorrelation
4. Built OLS and Geographically Weighted Regression models
5. Trained and evaluated Random Forest, XGBoost, and SVR models via `mlspatial`
6. Integrated disease burden with service availability and generated reproducible reports

Next steps for the Ghana R User Community: - Replace the built-in Africa data with Ghana Health Service district-level data. - Download real WorldClim rasters via `geodata::worldclim_global()`. - Explore `mlspatial` vignettes: `vignette("mlspatial")` after installing the package.

Reference: Azeez, A., & Noel, C. (2025). *Predictive Modelling and Spatial Distribution of Pancreatic Cancer in Africa Using Machine Learning-Based Spatial Model*. Zenodo. <https://doi.org/10.5281/zenodo.16529986>